

NOTES ON CATEGORICAL SYSTEMS THEORY

MATTEO CAPUCCI
UNIVERSITY OF STRATHCLYDE

APRIL 30, 2025

ABSTRACT. Categorical systems theory (CST) is a doubly-categorical framework for approaching and developing structural theories of systems and behaviour of any kind, emphasizing compositionality and functoriality. These notes are meant to be (1) a basic reference for CST and (2) a source of examples and working developments. They're under active development, so quite far from camera-ready, and possibly never complete.

CONTENTS

1. Introduction	1
1.1. A bit of history	2
1.2. What CST is all about.	3
1.3. Paradigms, doctrines, theories.	4
2. Composition	6
3. Systems	9
3.1. Doctrines of systems	19
4. Behaviours	19
4.1. Generalized behaviours	26
4.2. Doctrines of behaviour	26
5. Time	27
5.1. Locality	29
5.2. Open maps	29
6. Relation with coalgebraic systems theory	29

1. INTRODUCTION

Systems are ubiquitous, in science as in life. People regularly deal with physical systems, political systems, economical systems, living systems, learning systems, writing systems, voting systems, computing systems, etc. As we zoom into a thing, we inevitably realize it is comprised of smaller interacting parts. As we zoom out, we realize it is itself part of an even more complex system.

Given the staggering variety systems come in, it is no surprise the existing scientific and mathematical frameworks to describe them are manifold and often incompatible with one another. Categorical systems theory (CST) does not replace for the disciplines which study these particular

systems, nor the theoretical toolkits that scientists developed for them. Instead, CST gives a bird eye view of general systems theory and can thus organize, inform and comment on design decisions of different mathematical specifications of systems.

An important consequence of this approach is *removing opacity*. In fact, each framework created to deal with systems makes certain assumptions regarding the structure of the systems under scrutiny: the way they compose, the way they relate, which behaviours or aspects of the systems are of interest and what even means to display a certain behaviour. It's easy to get lost in these questions, and to miss important insight because of *blindness to structure*.¹ An important role CST fulfils is clarifying, within each individual framework, which choices have been made and why: this is only possible if the alternatives can also be compared.

Instead of espousing a specific mathematical specification of systems, CST predicates upon the general features such specifications should have. This allows the language of CST to be uniform across a wild variety of systems, from Petri nets to partial differential equations, from finite state machines to probabilistic automata.

1.1. A bit of history. Categorical approaches to general or particular systems theory have been around for a long time. Here we try to give a short chronology, and we will gather examples as we come across them. However, the reader should not expect a complete history of the field from this section, but rather a list of pointers to the literature. The main aim of the following story is to acknowledge the shoulders of the giants we stand on, and show how the ideas converging in DCST often arisen somewhere else first.

The earliest example of categorical systems theory is probably due to Rosen [Ros78], which deals with systems from a behavioural point of view. The work pioneers the idea of gathering systems in categories and considering their observable behaviour as input-output relations.

In the 70s, lots of automata theory was also interpreted categorically, generalizing it considerably, in the works of Arbib and Manes [AZ69; AM74b; AM74a; AM75] and Goguen [Gog72b; Gog72a; Gog73].

In the 90s, a series of papers by Katis, Sabadini, Walters and others established a theory of machines [SW93; BSW96; KSW97a; KSW97b; KSW99; KSW02] which anticipated some of the ideas regarding functorial semantics of behaviour, finding categories of matrices are well-adapted to receive such functors.

The works on structured and decorated cospans [FS07; Fon15; BM20; BC20; BCV22] deals with the doctrine of port-plugging systems (circuits) in modern form, and started using some ideas from double category theory to talk about them. This also happens in [Ler18; CGKS20]. Notably, Courser has been the first (to our knowledge) to explicitly recognize double categories as instrumental to the compositional study of open systems, see [Cou20].

It is Spivak [Spi13] who first introduced the operadic view of systems, see also Libkind's [LBPF21] for a more mature take.

Finally, coalgebraic automata theory [Trn77; Rut00; KV08; Jac17] can be considered the currently most mature form of 'general system theory'. Contrary to other examples so far,

¹Also known as the fish-in-water phenomenon: we get so used to certain aspects of our setting we have a hard time realizing their role and significance.

coalgebraic automata theory mostly overlooked composition of systems and focused instead on categories *of* systems. However, it has a large and well-developed literature covering an immense breadth of topics, and delving deeply in many applications such as model checking and semantics of modal logic [Mos99].

Thus we can say that albeit the current form and conceptualization of CST (using the technology of doubly indexed categories) is due to David Jaz Myers, various pieces of this philosophy have been around for far longer, as well as specific incarnations of them.

At the moment, most of CST lives in Myers' own book [Mye22], itself a longer version of the shorter preprint [Mye20c] (where the notion of *doctrine* wasn't yet developed). Moreover, Myers has given a few talks about the topic in the past years:

- (1) D. J. Myers. *A general definition of open dynamical system*. 2020. URL: <https://www.youtube.com/watch?v=8T-Km3taNko>
- (2) D. J. Myers. *Open dynamical systems, trajectories and hierarchical planning*. 2020. URL: <https://www.youtube.com/watch?v=3FxeY5DbPn0>
- (3) D. J. Myers. *Double Categories of Open Dynamical Systems*. 2020. URL: https://www.youtube.com/watch?v=f9fjf9lo2_M
- (4) D. J. Myers. *Paradigms of composition*. 2021. URL: <https://www.youtube.com/watch?v=50s62D5Ah-M>

I also gave a talk about CST and its extension to cybernetic systems:

- (5) M. Capucci. *From categorical systems theory to categorical cybernetics*. 2022. URL: <https://www.youtube.com/watch?v=wtgfyjFIHBQ>

1.2. What CST is all about. In Greek, the word ‘system’ means ‘composite’. Categorical systems theory takes this etymology very seriously, approaching the study of systems as the study of ‘things that compose’. Thankfully, the mathematical theory of composition is rich and has been abundantly studied before, under the guise of **operads**. The idea that systems are algebras of operads is more than a decade old now, being first proposed by Spivak in his 2013 paper [Spi13].

Remark 1.0.1. The word operad is quite overloaded, and, in some sense, not overloaded enough. Traditionally, an operad is a structure encoding formal operations of arbitrary finite arity which compose associatively and have a unit. In fact, the idea can be easily generalized much further, by having “arities” being structured objects. In this generalized form, operads are usually called *multicategories*, but I’d like to keep calling them operads because (a) morally, they still are and (b) operad is a much nicer and less scary word than multicategory. This translates to their even-more-generalized form, *T*-multicategories, which I call *T*-operads.

Thus, if *operads* are **theories of composition** (and they are), *theories of systems* should be algebras of theories of compositions.

This is the *algebraic* aspect of systems theory: it concerns the way systems are put together by operations (incidentally, also the word *algebra* is etymologically related to composition)! There is also a *geometric* aspect to systems theory, if we might abuse the algebro-geometric duality. Systems are objects with an internal structure, which can be probed by morphisms

which compare systems to each other. Having this extra geometric structure is quite important, albeit often overlooked. It is not overlooked in coalgebraic automata theory, where the algebraic aspect is neglected but the idea that systems shall be objects of a category is taken in great consideration.

Therefore, *theories of systems* should be algebras of *double operads*, i.e. operads in categories.

Remark 1.0.2. This ‘definition’ is preemptively general. While the ideal is to eventually work in terms of general T -operads (that being the ‘morally right’ setting), at the minute most of categorical systems theory is done for $T = \mathbf{smc}$, the *free symmetric monoidal category 2-monad* on $\mathbf{Cat}$ (Example 4.1.16 in [Lei04]). Concretely, this means that a double $\mathbf{smc}$ -operad is a *symmetric monoidal double category*.

1.3. Paradigms, doctrines, theories. The cosmogony of categorical systems theory can be organized in three levels of decreasing abstraction. It’s easier to start from the topmost level:

Definition 1.1 (Paradigm). A **paradigm** of systems theory is a way to answer the following questions:

- (1) What kind of *comparisons* between systems we want to ponder?
- (2) What kind of *compositions* of systems we want to ponder?

The most familiar paradigms in applied category theory are the following:

Example 1.1 (Paradigm of sets). In this paradigm, systems are organized in sets, thus can only be compared for equality. Composition is described by symmetric operads, thus basically symmetric monoidal categories. This is a fairly common paradigm in the literature, e.g. Spivak’s paper on wiring diagrams [Spi13] can be considered to work in the paradigm of sets.

Example 1.2 (Paradigm of categories). In this paradigm, systems are organized in categories, thus can be compared with morphisms. Composition is described by symmetric double operads, thus basically symmetric monoidal double categories. **This is the default paradigm in categorical systems theory.**

One could conceive other paradigms. For instance, one might want to compare systems by quantifying their similarity with a number, a cohomology class, or some other extensive measurement. One could compose them in different ways, for instance by glueing them instead of wiring them.

Mathematically, the answers to the questions posed by a choice of paradigm correspond to the following:

Definition 1.2 (Paradigm). A **paradigm** is an equipment $\mathbb{E}$ along with a monad $T : \mathbb{E} \rightarrow \mathbb{E}$, i.e. a way to define what ‘operad’ and ‘algebra’ mean.

Once fixed a paradigm, we can build a 2-category of theories, whose objects are theories of systems and whose maps are lax maps thereof.

Definition 1.3 (2-category of theories). Let $(\mathbb{E}, T)$ be a paradigm. The associated **2-category of theories** $\mathbf{Th}^{(\mathbb{E}, T)}$ is the 2-category of T -operads and right algebras thereof, with *lax* maps and 2-cells. Objects are thus pairs $(\mathbb{C}, \mathbf{Sys})$ where $\mathbb{C}$ is a T -operad and $\mathbf{Sys}$ a right algebra thereof.

Definition 1.4 (Theory). A **theory** for a paradigm $(\mathbb{E}, T)$ is an object of $\mathbf{Th}^{(\mathbb{E}, T)}$.

Example 1.3 (Theories in the set paradigm). The 2-category of theories for the paradigm $(\mathbf{Set}, \mathbf{sm})$ is the 2-category whose objects are pairs $(\mathcal{C}, \mathbf{Sys})$ where the first is a symmetric monoidal category and the latter is a symmetric monoidal copresheaf $\mathbf{Sys} : \mathcal{C} \rightarrow \mathbf{Set}$. A map of theories is given by a symmetric lax monoidal functor between the base categories and a natural transformation.

Example 1.4 (Theories in the categories paradigm). The 2-category of theories for the paradigm $(\mathbf{Cat}, \mathbf{smc})$ is the 2-category whose objects are pairs $(\mathbb{C}, \mathbf{Sys})$ where the first is a symmetric monoidal double category and the latter is a symmetric monoidal lax copresheaf $\mathbf{Sys} : \mathbb{C} \rightarrow \mathbf{Set}$, also known as a doubly indexed category. A map of theories is given by a symmetric lax monoidal lax double functor between the base double categories and a lax natural transformation.

However, the concept of ‘theory’ at the minute is underspecified. Most times we describe a theory we are actually giving a description of class of theories all parametrized by some common data (e.g. a category with pullbacks, a category together with a monad, etc.). So a theory is often just some data we can use to get an operad and an algebra in a specified way. Informally, one defines a doctrine as follows (this one is straight from Myers’s book [Mye22]):

Informal Definition 1 (Doctrine). A **doctrine** of systems is a particular way to answer the following questions about it means to be a systems theory:

- (1) What does it mean to be a system? Does it have a notion of states, or of behaviors? Or is it a diagram describing the way some primitive parts are organized?
- (2) What should the interface of a system be?
- (3) How can interfaces be connected in composition patterns?
- (4) How are systems composed through composition patterns between their interfaces?
- (5) What is a map between systems, and how does it affect their interfaces?
- (6) When can maps between systems be composed along the same composition patterns as the systems?

Thus a doctrine is a *uniform*, meaning *functorial*, way of building theories:

Definition 1.5 (Doctrine). A **doctrine** $\mathcal{D}\mathbf{oc}$ in the paradigm $(\mathbb{E}, T)$ is a 2-functor $\mathbf{Sys}^{\mathcal{D}\mathbf{oc}} : \mathbf{Th}^{\mathcal{D}\mathbf{oc}} \longrightarrow \mathbf{Th}^{(\mathbb{E}, T)}$. The objects of $\mathbf{Th}^{\mathcal{D}\mathbf{oc}}$ are called **theories for the doctrine** $\mathcal{D}\mathbf{oc}$.

Remark 1.5.1. The reason we already called $\mathbf{Th}^{(\mathbb{E}, T)}$ the 2-category of *theories* is easily seen: clearly the identity functor of $\mathbf{Th}^{(\mathbb{E}, T)}$ is a doctrine, and in fact the ‘universal one’, since it is terminal among doctrines over $(\mathbb{E}, T)$. Thus all right algebras for T -operads in $\mathbb{E}$ are theories for the universal doctrine for the paradigm $(\mathbb{E}, T)$.

The definitive definition of theory mentions directly the doctrine:

Definition 1.6 (Theory). A **theory** $\mathbf{Sys}$ for a doctrine $\mathcal{D}\mathbf{oc}$ is an object in $\mathbf{Th}^{\mathcal{D}\mathbf{oc}}$.

2. COMPOSITION

The starting point for defining a theory of systems is defining a double category of composition patterns such systems use to interact with one another. In practice, defining the systems and

defining the composition patterns are activities that influence one another. Systems are made out of composition patterns themselves, and it's usually them we have in mind when we approach a formalization problem. So one often starts by asking how the systems at hand could possibly be composed together.

Composition patterns usually form an operad.² Operads are indeed ways to specify how ‘small things fit into larger things’, i.e. they are *theories of composition*. Most importantly, they give meaning to various kinds of wiring diagrams [Spi13; VSL14; LBPF21].

One can also see composition patterns as *processes* that extend a given system with further dynamics, possibly gathering many systems in one. This point of view can be more natural from a ‘European’ point of view, more acquainted with string diagrams rather than wiring diagrams. In fact one can see double categories of composition patterns as higher-dimensional extensions of the process theories of Abramsky, Coecke, Gogioso [AC04; CK18].

Definition 2.1. A **composition theory** is a symmetric monoidal double category with attitude.

Hence any monoidal double category can be a composition theory if we have a convincing interpretation of it as such. At this level of generality there's no justification for limiting this definition further. Of course any specific doctrine of systems can make more opinionated choices on which monoidal double categories to admit, and we will see plenty of examples of this.

Remark 2.1.1. Rather than defining double categories of processes to be monoidal, one should arguably define them to be multicategories. Thus a nicer, if more exotic, definition of composition theory would be that of a ‘multicategory internal to categories’. This is the reason here we often denote processes as having many inputs. With our definitions, one might interpret the notation $I_1, \dots, I_n$ as denoting a monoidal product, i.e. we denote with ‘,’ the monoidal product on processes.

A composition theory $\mathbb{C}$ thus unpacks as follows:

- (1) There are a number of objects $I, J, K, \dots$ which are **interfaces** of the processes.
- (2) There are tight maps $h : I \longrightarrow J$ which are **maps of interfaces**, without dynamic content, they simply compare different interfaces with each other.³

$$I \xrightarrow{h} J \tag{2.1}$$

- (3) There are loose maps $p : I_1, \dots, I_n \dashrightarrow K$ which are the **processes** or **compositions**, connecting interfaces in some way.

$$\begin{array}{c} I_1, \dots, I_n \\ \downarrow p \\ K \end{array} \tag{2.2}$$

²By ‘operad’ we mean ‘coloured operad’ which means ‘multicategory’.

³One could call these ‘algebraic maps of interfaces’, as their role is to provide morphisms that *preserve* the interface structure, in order to compare them.

- (4) There are squares $\alpha : p \Rightarrow q$ which are **maps of processes** supported by given maps of interfaces:

$$\begin{array}{ccc}
 I_1, \dots, I_n & \xrightarrow{h_1, \dots, h_n} & J_1, \dots, J_n \\
 \downarrow p & \xRightarrow{\alpha} & \downarrow q \\
 K & \xrightarrow{k} & L
 \end{array} \tag{2.3}$$

Moreover, squares and 1-cells compose both vertically and horizontally, with vertical composition associative and unital only up to coherent isomorphism, see [Gra19].

Notice this definition distinguishes between two kinds of 1-cells involved in the study of open processes: the 1-cells which act *as* processes and the 1-cells acting as *morphisms of* processes. In fact, the usual yoga of process theories (which amount to symmetric monoidal ‘single’ categories) only focuses on the compositional properties of processes (they compose like morphisms). But in category theory we know that if we want to study something, we need to study morphisms between them!

Example 2.1 (Alphabets). Let $\mathbf{Alph} = \mathbf{FinSet}^\uparrow$ be the double category of alphabets and alphabet reductions, and maps thereof. Its objects are finite sets of symbols we call ‘alphabets’. Its tight maps are maps of finite sets. Its loose maps are *alphabet reductions*, which are map of finite sets in the opposite direction:

$$p : \Sigma \dashrightarrow \Sigma' \in \mathbf{FinSet}(\Sigma', \Sigma) \tag{2.4}$$

Squares are commutative squares:

$$\begin{array}{ccc}
 \Sigma & \xrightarrow{h} & \Xi \\
 \uparrow p & & \uparrow q \\
 \Sigma' & \xrightarrow{k} & \Xi'
 \end{array} \tag{2.5}$$

Example 2.2 (Bidirectional processes). Consider the following double category of *deterministic bidirectional processes* $\mathbb{L}ens$:

- (1) interfaces are given by pairs or sets $\begin{pmatrix} A^- \\ A^+ \end{pmatrix}$, whose monoidal product is given by component-wise product,
- (2) processes $\begin{pmatrix} A^- \\ A^+ \end{pmatrix} \rightleftharpoons \begin{pmatrix} C^- \\ C^+ \end{pmatrix}$ are given by **lenses** $\begin{pmatrix} f^\sharp \\ f \end{pmatrix}$, comprised of a *get part* $f : A^+ \rightarrow C^+$ and a *put part* $f^\sharp : A^+ \times C^- \rightarrow A^-$,
- (3) maps of interfaces $\begin{pmatrix} A^- \\ A^+ \end{pmatrix} \rightrightarrows \begin{pmatrix} B^- \\ B^+ \end{pmatrix}$ are given by **charts** $\begin{pmatrix} g^\flat \\ g \end{pmatrix}$, comprised of a *states part* $g : A^+ \rightarrow B^+$ and a *directions part* $g^\flat : A^+ \times A^- \rightarrow B^-$,
- (4) behaviours are given by arrangements

$$\begin{array}{ccc}
 \begin{pmatrix} A^- \\ A^+ \end{pmatrix} & \xrightleftharpoons[g]{g^\flat} & \begin{pmatrix} B^- \\ B^+ \end{pmatrix} \\
 \downarrow f \uparrow f^\sharp & & \downarrow k \uparrow k^\sharp \\
 \begin{pmatrix} C^- \\ C^+ \end{pmatrix} & \xrightleftharpoons[h]{h^\flat} & \begin{pmatrix} D^- \\ D^+ \end{pmatrix}
 \end{array} \tag{2.6}$$

lenses	category	F
deterministic	$\mathcal{C}$ cartesian monoidal	$\mathbf{Set}/_{\text{proj}}-$ (or $\mathbf{coKl}(-\times=)$)
deterministic (dependent)	$\mathcal{C}$ finite limits	$\mathbf{Set}/-$
possibilistic	$\mathcal{E}$ topos	$\mathbf{biKl}(-\times=, \mathcal{P})$ ($\mathcal{P}$ powerset monad)
probabilistic	$\mathbf{Msbl}$	$\mathbf{biKl}(-\times=, \Delta)$ (Δ probability monad)
effectful	$\mathcal{C}$ cartesian monoidal	$\mathbf{biKl}(-\times=, M)$ (M commutative monad)
differential (Euclidean)	$\mathbf{Euc}$	$\mathbf{Euc}/_{\text{subm}}-$
differential (general)	$\mathbf{Smooth}$	$\mathbf{Smooth}/_{\text{subm}}-$

Table 1. *Flavours of F -lenses. We put them in pairs, with the first being ‘simple’ lenses, like those in Example 2.2, and the second ‘dependent’.*

such that for every $a^+ \in A^+$ and $c^- \in C^-$:

$$\begin{aligned} k(g(a^+)) &= h(f(a^+)), \\ g^b(a^+, f^\sharp(a^+, c^-)) &= k^\sharp(g(a^+), h^b(f(a^+), c^-)). \end{aligned} \tag{2.7}$$

The last conditions are hard to parse formally, but basically they say that both squares one can spot in (2.6) commute. Concretely, we have two bidirectional processes $\begin{pmatrix} f^\sharp \\ f \end{pmatrix}$ and $\begin{pmatrix} k^\sharp \\ k \end{pmatrix}$ and a way to map between their interfaces. We are then asking that their dynamics commute with such maps.

Remark 2.1.2. Viewed as composition patterns, lenses are algebras of the operad of wiring diagrams [Spi13]. Thus they represent ways to wire a number of boxes into a larger box:

Matteo: wiring diagram

Example 2.3. The previous example can be generalized greatly by employing F -lenses [Spi19]. These are lenses in which the backward part is dependent on the forward part in a way specified by an indexed category F . Intuitively, this correspond to a wiring pattern which can change depending on the what flows in the wires (see [Spi20]).

The definitions of F -lenses and F -charts are substantially identical to the ones above, as well as that for the squares (except the ‘commutativity condition’ is now harder to eyeball). We gather some examples here:

Matteo: add dependent variants for other lenses

Example 2.4 (Behavioural theory). Given any category with finite limits $\mathcal{E}$, there is a cartesian double category $\mathbf{Span}(\mathcal{E})$. Here, the objects are the same of $\mathcal{E}$, as well as the tight maps. The

loose maps are given by spans instead, and the squares by morphisms of spans:

$$\begin{array}{ccc}
 I & \xrightarrow{h} & J \\
 p_\ell \uparrow & & \uparrow q_\ell \\
 X & \xrightarrow{\alpha} & Y \\
 p_r \downarrow & & \downarrow q_r \\
 I' & \xrightarrow{k} & J'
 \end{array} \tag{2.8}$$

We call these double categories ‘behavioural theories of processes’ since they model processes only by their input-output (proof-relevant) relation, hence by the behaviour they expose.

Example 2.5 (Generalized behavioural theory). Let $\mathcal{E}$ be a cartesian category with strong display maps $\mathcal{D}$ ([Jac99, Definition 10.4.1]), meaning $\mathcal{D}$ is a wide subcategory of maps (called *display maps* and denoted as $\rightarrow$) which is closed under pullback along arbitrary maps and under products (implying product projections are display). We can then form a double category $\mathbf{Span}(\mathcal{D}, \mathcal{E})$ whose objects and tight maps are the same of $\mathcal{E}$, and whose loose maps are given by spans whose left leg is display

$$\begin{array}{ccc}
 I & \xrightarrow{h} & J \\
 p_\ell \uparrow & & \uparrow q_\ell \\
 X & \xrightarrow{\alpha} & Y \\
 p_r \downarrow & & \downarrow q_r \\
 I' & \xrightarrow{k} & J'
 \end{array} \tag{2.9}$$

3. SYSTEMS

Systems are the things compositions link, or the things composition patterns compose.

Definition 3.1. A **doubly indexed category**, or **action of a double category**, is given, informally, by a unitary lax double functor $\mathbf{Sys} : \mathbb{C}^\top \rightarrow \mathbf{Cat}$.

We recall $\mathbf{Cat}$ is the cartesian monoidal double category of categories, functors, profunctors and natural transformations. A unitary lax functor is a double functor preserving vertical identities strictly but not vertical composition. Hence given two maps of interfaces $h : I \rightarrow J$, $k : J \rightarrow K$, we have a natural transformation $\ell_{h,k} : \mathbf{Sys}(h) \otimes \mathbf{Sys}(k) \rightarrow \mathbf{Sys}(h \circ k)$ between profunctors.

Definition 3.2. A **theory of systems over the composition theory** $\mathbb{C}$ is a monoidal doubly indexed category $\mathbf{Sys} : \mathbb{C}^\top \xrightarrow{\text{unitary lax}} \mathbf{Cat}$ with attitude.

Concretely, $\mathbf{Sys}$ maps interfaces to **categories of systems**, compositions to **composition functors**, maps of interfaces to **mapping profunctors** and maps of compositions to **composition transformations**.

Hence given an interface $I : \mathbb{C}$, we think of the objects of $\mathbf{Sys}(I)$ as systems of a certain kind while the maps are **simulations** between them, i.e. some notion of structure-preserving map between them.

$$\mathbf{Sys}(I) = \left\{ S \xrightarrow{\varphi} T \right\} \tag{3.1}$$

The functors induced by a composition act by extending a system with that composition. If we think of the composition as a composition pattern instead, the functor assembles in a composite system:

$$\mathbf{Sys}(I \xrightarrow{p} K) : \mathbf{Sys}(I) \longrightarrow \mathbf{Sys}(K) \quad (3.2)$$

The profunctors induced by a map of interfaces give notions of simulations between systems on different interfaces:

$$\mathbf{Sys}(I \xrightarrow{h} J) : \mathbf{Sys}(I) \leftrightarrow \mathbf{Sys}(J) \quad (3.3)$$

Hence an element $\ell \in \mathbf{Sys}(I \xrightarrow{h} J)(\mathcal{S}, \mathcal{T})$ is a *simulation of S in $\mathcal{T}$ mediated by the maps of interfaces h* .

Finally, squares in $\mathbb{C}$ induce squares witnessing the composition of a simulation of systems along a map of compositions:

$$\mathbf{Sys}(p \xrightarrow{\alpha} q) : \mathbf{Sys}(h) \Rightarrow (\mathbf{Sys}(p), \mathbf{Sys}(q))^* \mathbf{Sys}(k) \quad (3.4)$$

Example 3.1 (Closed dynamical systems). The most basic model of dynamical systems in mathematics is simply endomorphisms $\delta : S \rightarrow S$ on some space S in a category of ‘spaces’ $\mathcal{S}$. These systems are closed: they expose nothing of their state, and their dynamics can’t be influenced by external input: their composition theory is trivial! Consequently, the systems theory of closed dynamical systems is given by a single category $\mathbf{DynSys}_{\mathcal{S}}$:

$$\mathbf{DynSys}_{\mathcal{S}} : 1^{\top} \xrightarrow{\text{unitary lax}} \mathbf{Cat}. \quad (3.5)$$

In this category, the objects are endomorphisms and the maps are commuting squares of the form:

$$\begin{array}{ccc} S & \xrightarrow{\varphi} & R \\ \delta \downarrow & & \downarrow \gamma \\ S & \xrightarrow{\varphi} & R \end{array} \quad (3.6)$$

Clearly $\mathbf{DynSys}_{\mathcal{S}}$ is monoidal if $\mathcal{S}$ is. Thus given a category $\mathcal{S}$ of spaces, one gets a systems theory of **closed dynamical systems** in $\mathcal{S}$.

Example 3.2. There’s many possible variations on the definition of $\mathbf{DynSys}_{\mathcal{S}}$. Two that encompass many interesting examples are as follows.

- (1) One can choose an endofunctor $F : \mathcal{S} \rightarrow \mathcal{S}$ and consider F -coalgebras instead of mere endomorphisms as the dynamical systems. In this way one can get, e.g. non-deterministic closed systems. We denote this category $\mathbf{Coalg}(F)$. The basic case is recovered for the choice $F = 1_{\mathcal{S}}$.
- (2) One can choose a monoid of ‘time’ T and consider T -actions instead of mere endomorphisms. Notice one can pick $T : \mathbf{Mon}(\mathcal{S})$ but also $T : \mathbf{Mon}(\mathbf{Set})$, and then consider T -actions to be functors $BT \rightarrow \mathcal{S}$. In this way one can get, e.g. continuous time dynamical systems by choosing $T = \mathbb{R}$. We denote this category $\mathbf{TimeSys}(T)$. The basic case is recovered for the choice $T = \mathbb{N}$.

These two examples also admit a more interesting theory of compositions, as we are going to see shortly.

Example 3.3 (Monoid actions). The categories $\mathbf{TimeSys}(T)$ naturally gather in a doubly indexed category where the indexing double category is $\mathbf{Mon}(\mathcal{S})^\uparrow$. This is the double category of monoids in $\mathcal{S}$ and commutative squares thereof, except we take the opposite of the vertical direction. Hence a vertical morphism $p : M \dashrightarrow N$ corresponds to a morphism of monoids $N \rightarrow M$.

This double category is a composition theory for the theory of dynamical systems ‘with time’ described above. In fact a vertical morphism $p : M \dashrightarrow N$ maps to a functor $\mathbf{TimeSys}(M) \rightarrow \mathbf{TimeSys}(N)$ given by ‘restriction of scalars’:

$$M \times S \xrightarrow{\delta} S \quad \longmapsto \quad N \times S \xrightarrow{p \times S} M \times S \xrightarrow{\delta} S, \quad (3.7)$$

and maps of monoids $h : M \rightarrow N$ also map to profunctors $\mathbf{TimeSys}(M) \leftrightarrow \mathbf{TimeSys}(N)$ that sends a pair of dynamical systems $S : \mathbf{TimeSys}(M)$, $R : \mathbf{TimeSys}(N)$ to the set of squares

$$\begin{array}{ccc} M \times S & \xrightarrow{h \times \varphi} & N \times R \\ \delta \downarrow & & \downarrow \gamma \\ S & \xrightarrow{\varphi} & R \end{array} \quad (3.8)$$

Example 3.4 (Labelled transition systems). We can use the double category $\mathbf{Alph}$ of alphabets defined in Example 2.1 to index labelled transition systems. When $T = \Sigma^*$ (the free monoid on a finite set Σ), then $\mathbf{DynSys}_{\mathcal{S}}(\Sigma^*)$ is the category of transition systems labelled with Σ . Hence we define $\mathbf{Trans} : \mathbf{Alph}^\top \xrightarrow{\text{unitary lax}} \mathbf{Cat}$ by restricting $\mathbf{DynSys}_{\mathbf{Set}}$ along the double functor $(-)^* : \mathbf{Mon}^\uparrow \rightarrow \mathbf{Alph}$ given by taking free monoids.

Example 3.5 (Finite state machines). A finite states machine with alphabet $\Sigma : \mathbf{FinSet}$ is a finite set S of *states* equipped with a *transition function* $\delta : S \times \Sigma \rightarrow S$ and a subset $S^* \subseteq S$ of *accepting states*. In other words, it’s a labelled transition system with alphabet Σ equipped with a predicate S^* over S . These form a systems theory over $\mathbf{Alph}$, which mostly behaves like $\mathbf{Trans}$. The only substantial difference is that morphisms of finite states machines are given by commutative squares like

$$\begin{array}{ccc} S \times \Sigma & \xrightarrow{\varphi} & R \times \Sigma \\ \delta \downarrow & & \downarrow \gamma \\ S & \xrightarrow{\varphi} & R \\ \cup & & \cup \\ S^* & \subseteq & R^* \end{array} \quad (3.9)$$

where the commutativity of the bottom square means $\varphi(S^*) \subseteq R^*$.

Thus there is a theory of systems, the **theory of finite states machines**

$$\mathbf{FSM} : \mathbf{Alph}^\top \xrightarrow{\text{unitary lax}} \mathbf{Cat}. \quad (3.10)$$

Remark 3.2.1. Often FSMs are assumed to be equipped with an initial state too. Adding this data to the previous definition is straightforward, but we think doing so is less elegant.

Remark 3.2.2. In $\mathbf{Set}$ (or really, any topos $\mathcal{E}$) FSMs over an alphabet Σ can be seen, alternatively, as coalgebras for endofunctor $2 \times (-)^\Sigma$. Thus one can instantiate the machinery of Example 6.1

to get categories of systems. There is a subtle problem with that though: morphisms of FSMs naturally make use of the order structure on 2 , which is not really accounted for in the ‘vanilla’ coalgebraic framework. Hence we believe it’s closer to the nature of FSMs to consider them as transition systems equipped with a subobject of states.

Example 3.6 (Moore machines). Moore machines are open dynamical systems. Myers spends a long time developing their theory in [Mye22]. In fact a very large class of systems is captured by Moore machines, especially in the extended form Myers considers.

‘Classical’ Moore machines have a set of states S , a set of inputs I , a set of outputs O , and two maps $\text{expose} : S \rightarrow O$ and $\text{update} : S \times I \rightarrow S$. Hence they are given by lenses of the form $\begin{pmatrix} S \\ S \end{pmatrix} \rightleftarrows \begin{pmatrix} I \\ O \end{pmatrix}$. The special form of the left boundary is what makes them system-y: we guarantee statefulness of by guaranteeing the two ‘ S ’ on the left are the same. We do so by defining maps of Moore machines (over a fixed interface $\begin{pmatrix} I \\ O \end{pmatrix}$) to be squares in $\mathbf{Lens}$ (Example 2.2) of the form:

$$\begin{array}{ccc}
 \begin{pmatrix} S \\ S \end{pmatrix} & \begin{array}{c} \xrightarrow{\pi_2 \circ \varphi} \\ \xrightarrow{\varphi} \end{array} & \begin{pmatrix} R \\ R \end{pmatrix} \\
 \text{update}_S \updownarrow \text{expose}_S & & \text{update}_R \updownarrow \text{expose}_R \\
 \begin{pmatrix} I \\ O \end{pmatrix} & \xlongequal{\quad} & \begin{pmatrix} I \\ O \end{pmatrix}
 \end{array} \tag{3.11}$$

Notice the special form of the top chart: the map $\varphi : S \rightarrow R$ is used both on the bottom and on top. The fact maps of Moore machines cannot distinguish between top and bottom is the reason that boundary behaves as a stateful interface.

The ‘generalized’ Moore machines of Myers are obtained by passing from lenses to F -lenses. This testifies an important conceptual shift.

Fix a category $\mathcal{C}$ and an indexed category $F : \mathcal{C}^{\text{op}} \rightarrow \mathbf{Cat}$. First of all, F -lenses are now bidirectional morphisms that have some kind of mode-dependence: an object $\begin{pmatrix} I \\ O \end{pmatrix}$ in $\mathbf{Lens}_F$ is given by an object $O : \mathcal{C}$ and an object $I : F(O)$. This can be thought as a type dependent on O or as a bundle over O . The important thing is that we introduce a more elaborate (and thus expressive) model of bidirectionality: stuff happens in the forward/bottom part (given by a morphism in $\mathcal{C}$), and then, *depending on what happened there*, something else happens in the backward/top part. The difference can be substantial, to the point of having completely different types involved.

In generalized Moore machines, we use this added dependency to distinguish between *states* and *state changes*. We thus think of the two S s in $\begin{pmatrix} S \\ S \end{pmatrix}$ as playing different roles: the bottom one is a proper object of states, whereas the top one is actually more accurately represented by something like $TS : F(S)$, an object ‘over’ S (hence depending on S , varying with it) accounting for the possible changes in state.

Hence whereas in a classical Moore machines all states are potentially reachable from any other states, we now contemplate machines in which the way we change state can be more complex. For example, we might choose a distribution of new states, or allow transitions only to states ‘nearby’ in some sense (thus introducing geometric structure on S).

Still, we want to be coherent in our meaning of ‘change’. Hence we parametrize theories of generalized Moore machines by a section $T : \mathcal{C} \rightarrow \int F$ picking out uniformly, for each $S : \mathcal{C}$, an object of changes $\begin{pmatrix} TS \\ S \end{pmatrix}$ over it. Importantly, T also maps morphisms $\varphi : S \rightarrow R$ to morphisms of changes $\begin{pmatrix} T\varphi \\ \varphi \end{pmatrix} : \begin{pmatrix} TS \\ S \end{pmatrix} \rightarrow \begin{pmatrix} TR \\ R \end{pmatrix}$ (which we can think of as some kind of derivative). In this way we preserve the statefulness of the boundary.

Thus a theory of Moore machines is constructed as follows. Chosen $\mathcal{C} : \mathbf{Cat}$ monoidal, $F : \mathcal{C}^{\text{op}} \rightarrow \mathbf{Cat}$ lax monoidal and $T : \mathcal{C} \rightarrow \int F$ lax monoidal section, we first build the composition theory $\mathbb{L}ens_F$ of F -lenses and F -charts. Then, we define, for each $\begin{pmatrix} I \\ O \end{pmatrix} : \mathbb{L}ens_F$, the categories

$$\mathbf{Moore}_{(F,T)}\left(\begin{pmatrix} I \\ O \end{pmatrix}\right) = \left\{ \begin{array}{ccc} \begin{pmatrix} TS \\ S \end{pmatrix} & \xrightleftharpoons[g]{T\varphi} & \begin{pmatrix} R \\ R \end{pmatrix} \\ \text{update}_S \updownarrow \text{expose}_S & & \text{update}_R \updownarrow \text{expose}_R \\ \begin{pmatrix} I \\ O \end{pmatrix} & \xlongequal{\quad} & \begin{pmatrix} I \\ O \end{pmatrix} \end{array} \right\} \quad (3.12)$$

Given an F -lens $\begin{pmatrix} p^\sharp \\ p \end{pmatrix} : \begin{pmatrix} I \\ O \end{pmatrix} \rightrightarrows \begin{pmatrix} J \\ Q \end{pmatrix}$ we get a functor:

$$\mathbf{Moore}_{(F,T)}\left(\begin{pmatrix} p^\sharp \\ p \end{pmatrix}\right) : \mathbf{Moore}_{(F,T)}\left(\begin{pmatrix} I \\ O \end{pmatrix}\right) \longrightarrow \mathbf{Moore}_{(F,T)}\left(\begin{pmatrix} J \\ Q \end{pmatrix}\right)$$

$$\begin{array}{ccc} \begin{pmatrix} TS \\ S \end{pmatrix} & & \begin{pmatrix} TS \\ S \end{pmatrix} \\ \text{expose}_S \updownarrow \text{update}_S & \mapsto & \text{expose}_S \updownarrow \text{update}_S \\ \begin{pmatrix} I \\ O \end{pmatrix} & & \begin{pmatrix} I \\ O \end{pmatrix} \\ & & p \updownarrow p^\sharp \\ & & \begin{pmatrix} J \\ Q \end{pmatrix} \end{array} \quad (3.13)$$

Given an F -chart $\begin{pmatrix} h^b \\ h \end{pmatrix} : \begin{pmatrix} I \\ O \end{pmatrix} \rightrightarrows \begin{pmatrix} J \\ Q \end{pmatrix}$, the induced profunctor is

$$\mathbf{Moore}_{(F,T)}\left(\begin{pmatrix} I \\ O \end{pmatrix}\right)^{\text{op}} \times \mathbf{Moore}_{(F,T)}\left(\begin{pmatrix} J \\ Q \end{pmatrix}\right) \xrightarrow{\mathbf{Moore}_{(F,T)}\left(\begin{pmatrix} h^b \\ h \end{pmatrix}\right)} \mathbf{Set}$$

$$\begin{array}{ccc} \begin{pmatrix} TS \\ S \end{pmatrix} & \begin{pmatrix} TR \\ R \end{pmatrix} & \\ \text{update}_S \updownarrow \text{expose}_S & \text{update}_R \updownarrow \text{expose}_R & \\ \begin{pmatrix} I \\ O \end{pmatrix} & \begin{pmatrix} J \\ Q \end{pmatrix} & \end{array} \mapsto \left\{ \begin{array}{ccc} \begin{pmatrix} TS \\ S \end{pmatrix} & \xrightleftharpoons[\varphi]{T\varphi} & \begin{pmatrix} TR \\ R \end{pmatrix} \\ \text{update}_S \updownarrow \text{expose}_S & & \text{update}_R \updownarrow \text{expose}_R \\ \begin{pmatrix} I \\ O \end{pmatrix} & \xrightleftharpoons[h]{h^b} & \begin{pmatrix} J \\ Q \end{pmatrix} \end{array} \right\} \quad (3.14)$$

Finally, given a square

$$\begin{array}{ccc} \begin{pmatrix} I \\ O \end{pmatrix} & \xrightarrow[h]{h^b} & \begin{pmatrix} J \\ Q \end{pmatrix} \\ p^\# \uparrow \downarrow p & & q^\# \uparrow \downarrow q \\ \begin{pmatrix} I' \\ O' \end{pmatrix} & \xrightarrow[k]{k^b} & \begin{pmatrix} J' \\ Q' \end{pmatrix} \end{array} \quad (3.15)$$

in $\mathbb{L}ens_F$ we get a square in $\mathbb{C}at$

$$\begin{array}{ccc} \mathbf{Moore}\left(\begin{pmatrix} I \\ O \end{pmatrix}\right) & \xrightarrow{\mathbf{Moore}\left(\begin{pmatrix} p^\# \\ p \end{pmatrix}\right)} & \mathbf{Moore}\left(\begin{pmatrix} I' \\ O' \end{pmatrix}\right) \\ \downarrow \mathbf{Moore}\left(\begin{pmatrix} h^b \\ h \end{pmatrix}\right) & \xRightarrow{\mathbf{Moore}(\square)} & \downarrow \mathbf{Moore}\left(\begin{pmatrix} k^b \\ k \end{pmatrix}\right) \\ \mathbf{Moore}\left(\begin{pmatrix} J \\ Q \end{pmatrix}\right) & \xrightarrow{\mathbf{Moore}\left(\begin{pmatrix} q^\# \\ q \end{pmatrix}\right)} & \mathbf{Moore}\left(\begin{pmatrix} J' \\ Q' \end{pmatrix}\right) \end{array} \quad (3.16)$$

given by stacking squares:

$$\begin{array}{ccc} \mathbf{Moore}\left(\begin{pmatrix} h^b \\ h \end{pmatrix}\right)(S, R) & \xrightarrow{\mathbf{Moore}(\square)_{S,R}} & \mathbf{Moore}\left(\begin{pmatrix} h^b \\ h \end{pmatrix}\right)\left(\mathbf{Moore}\left(\begin{pmatrix} p^\# \\ p \end{pmatrix}\right)(S), \mathbf{Moore}\left(\begin{pmatrix} q^\# \\ q \end{pmatrix}\right)(R)\right) \\ \\ \begin{array}{ccc} \begin{pmatrix} TS \\ S \end{pmatrix} & \xrightarrow[\varphi]{T\varphi} & \begin{pmatrix} TR \\ R \end{pmatrix} \\ \uparrow \text{exposes}_S & & \uparrow \text{expose}_R \\ \begin{pmatrix} I \\ O \end{pmatrix} & \xRightarrow[h]{h^b} & \begin{pmatrix} J \\ Q \end{pmatrix} \\ \downarrow \text{update}_S & & \downarrow \text{update}_R \end{array} & \mapsto & \begin{array}{ccc} \begin{pmatrix} TS \\ S \end{pmatrix} & \xrightarrow[\varphi]{T\varphi} & \begin{pmatrix} TR \\ R \end{pmatrix} \\ \uparrow \text{exposes}_S & & \uparrow \text{expose}_R \\ \begin{pmatrix} I \\ O \end{pmatrix} & \xRightarrow[h]{h^b} & \begin{pmatrix} J \\ Q \end{pmatrix} \\ \downarrow \text{update}_S & & \downarrow \text{update}_R \\ \begin{pmatrix} I' \\ O' \end{pmatrix} & \xRightarrow[k]{k^b} & \begin{pmatrix} J' \\ Q' \end{pmatrix} \\ \uparrow p^\# & & \uparrow q^\# \end{array} \end{array} \quad (3.17)$$

We thus defined the **theory of (F, T) -generalized Moore machines**:

$$\mathbf{Moore}_{(F,T)} : \mathbb{L}ens_F^\top \xrightarrow{\text{unitary lax}} \mathbb{C}at. \quad (3.18)$$

Example 3.7 (Differential systems). A notable example of generalized Moore machines is given by differential open dynamical systems. These are Moore machines in **Smooth**, the category of smooth manifolds.⁴ We consider bundles given by submersions,⁵ since these are pullback-stable

⁴Sometimes it's useful to consider suitable generalizations, like diffeological or smooth spaces, to get better properties out of the theory. We don't go beyond undergraduate differential geometry here.

⁵A *submersion* of smooth manifolds is a surjective smooth map whose differential is also surjective. It means we are mapping to a manifold of equal or smaller dimension, and all the fibers are manifolds themselves (hence excluding critical points or singular submanifolds).

maps:

$$\mathbf{Smooth}/_{\text{subm}} - : \mathbf{Smooth}^{\text{op}} \longrightarrow \mathbf{Cat}$$

$$\begin{array}{ccccc} M & & \mathbf{Smooth}/_{\text{subm}} M & & M \xleftarrow{\quad} p^* E \\ \downarrow p & \mapsto & \uparrow p^* & & \downarrow \quad \quad \quad \downarrow \\ N & & \mathbf{Smooth}/_{\text{subm}} N & & N \xleftarrow{\quad \pi \quad} E \end{array} \quad (3.19)$$

We then choose the section $T : \mathbf{Smooth} \rightarrow \int \mathbf{Smooth}/_{\text{subm}}$ to be the one picking out, for each smooth manifold S , its tangent bundle $\pi_S : TS \rightarrow S$. This extends to smooth maps $\varphi : S \rightarrow R$ by mapping them to their differential, which is indeed a map of tangent bundles $TS \rightarrow TR$.

Then a Moore machine $\left(\begin{smallmatrix} \text{update} \\ \text{expose} \end{smallmatrix} \right) : \left(\begin{smallmatrix} TS \\ S \end{smallmatrix} \right) \rightleftharpoons \left(\begin{smallmatrix} I \\ O \end{smallmatrix} \right)$ is a pair of maps

$$\text{expose} : S \rightarrow O, \quad \text{update} : (s : S) \times (i : I(\text{expose}(s))) \rightarrow TS \quad (3.20)$$

which describe an observable on S and a non-autonomous vector field on it:

$$ds = \text{update}(s, i) \quad (3.21)$$

Notably, a system of this kind with a trivial interface is simply a vector field (i.e. a section of the tangent bundle).

Example 3.8 (Mealy machines). Similar to Moore machines are Mealy machines: they too are stateful open dynamical systems over a bidirectional interface $\left(\begin{smallmatrix} I \\ O \end{smallmatrix} \right)$, but their output is also dependent on their input:

$$\text{expose} : S \times I \rightarrow O, \quad \text{update} : S \times I \rightarrow S \quad (3.22)$$

Therefore they are usually given as a single map

$$\delta : S \times I \longrightarrow S \times O \quad (3.23)$$

in some cartesian monoidal category $(\mathcal{C}, 1, \times)$.

This seemingly inconsequential difference makes Mealy machines quite different. In fact, contrary to Moore machines, such systems cannot be covariantly reindexed by lenses anymore $\left(\begin{smallmatrix} p^\sharp \\ p \end{smallmatrix} \right) : \left(\begin{smallmatrix} I \\ O \end{smallmatrix} \right) \rightleftharpoons \left(\begin{smallmatrix} J \\ Q \end{smallmatrix} \right)$ like Moore machines, the problem arising when reindexing the input:

$$\mathbf{Mealy}_{\mathcal{C}} \left(\begin{smallmatrix} p^\sharp \\ p \end{smallmatrix} \right) : \delta \mapsto S \times J \xrightarrow{?} S \times I \xrightarrow{\delta} S \times O \xrightarrow{p} S \times Q \quad (3.24)$$

In fact in order to use $p^\sharp$ to convert a J to an I we need to have O in scope, but O can be *produced* only if I is available! So we run into a vicious cycle.

There's at least three ways to break this *empasse*. The first is to remove the dependency on O in $p^\sharp$. Hence instead of lenses we'll be looking at adapters, which are simply pairs of maps going in opposite directions.

The second is to solve the circularity by using a trace (or a trace-like) operator, thus asking for the ambient category $\mathcal{C}$ to be a traced or a feedback category (as in [KSW97a; DGR+21]). Then Mealy machines can be reindexed by morphisms in $\mathbf{Int}(\mathcal{C})$, which are indeed quite similar to Mealy machines:

$$p : \left(\begin{smallmatrix} I \\ O \end{smallmatrix} \right) \rightarrow \left(\begin{smallmatrix} J \\ Q \end{smallmatrix} \right) \equiv J \times O \rightarrow I \times Q. \quad (3.25)$$

These reindex δ by using the trace to get rid of extra O :

$$\mathbf{Mealy}_{\mathcal{C}}(p) : \delta \mapsto \text{trace}_O(S \times J \times O \xrightarrow{S \times p} S \times I \times Q \xrightarrow{\delta \times Q} S \times O \times Q). \quad (3.26)$$

They compose with each other similarly.

The third option is to use the loose opposite⁶ of the double category $\mathbf{Lens}(\mathcal{C})$. Indeed, Mealy machines are reindexed contravariantly by lenses:

$$\mathbf{Mealy}_{\mathcal{C}}\left(\begin{smallmatrix} p \\ p_{\sharp} \end{smallmatrix}\right) : \delta \mapsto S \times J \xrightarrow{\langle J, S \times p \rangle} J \times S \times I \xrightarrow{\delta \times J} J \times S \times O \xrightarrow{\langle S, p_{\sharp} \rangle} S \times Q \quad (3.27)$$

In fact, this works even with dependent lenses.

All these options (when well-defined) admit suitably dual, chart-like morphisms that fit into double categories of ‘commutative squares’, like the one in Example 2.2. This makes $\mathbf{Mealy}_{\mathcal{C}}$ a theory of systems:

$$\mathbf{Mealy}_{\mathcal{C}} : \mathbb{P}^{\top} \xrightarrow{\text{unitary lax}} \mathbf{Cat} \quad (3.28)$$

for $\mathbb{P} = \mathbf{Adap}(\mathcal{C})$ (adapters & pairs of morphisms), $\mathbf{Lens}(\mathcal{C})^{\text{lop}}$ (colenses & ‘cocharts’), $\mathbf{Int}(\mathcal{C})$ (**Int**-morphisms & their duals). Clearly the latter is admissible only when $\mathcal{C}$ is traced.

Example 3.9 (Behavioural theories, [Mye22, Definition 6.2.1.2]). Any behavioural theory of compositions $\mathbf{Span}(\mathcal{E})$ (Example 2.4) supports a behavioural theory of systems (recall $\mathcal{E}$ is assumed to be cartesian, hence admits all pullbacks):

$$\mathcal{E} : \mathbf{Span}(\mathcal{E})^{\top} \xrightarrow{\text{unitary lax}} \mathbf{Cat} \quad (3.30)$$

$$\mathcal{E}(I) = \left\{ \begin{array}{ccc} S & \xrightarrow{h} & R \\ \text{observes} \downarrow & & \downarrow \text{observer} \\ I & \xlongequal{\quad} & I \end{array} \right\} = \mathcal{E}/I \quad (3.31)$$

In this theory, a system S over the interface $I : \mathcal{E}$ is simply a state space $S : \mathcal{E}$ together with an observation $\text{observe}_S : S \rightarrow I$.⁷

There’s two ways to understand this map. The first is to think of it as a literal observable. Hence S is to be considered as a space of states while I as a space of quantities, and $\text{observe} : S \rightarrow I$ maps a state to its corresponding observable.

⁶Let a *colens* in $\mathcal{C}$ $\left(\begin{smallmatrix} I \\ O \end{smallmatrix}\right) \rightleftharpoons \left(\begin{smallmatrix} J \\ Q \end{smallmatrix}\right)$ be a pair of maps $p : J \rightarrow I$ and $p_{\sharp} : J \times O \rightarrow Q$. Both lenses and colenses embed in $\mathbf{Int}(\mathcal{C})$ when the latter exists, and both trivialize the tracing because they present only non-trivial circularity. All of adapters, lenses, colenses and $\mathbf{Int}(\mathcal{C})$ are categories of optics fit into a diamond:

$$\begin{array}{ccc} & \mathbf{Int}(\mathcal{C}) & \\ \nearrow & & \nwarrow \\ \mathbf{Colens}(\mathcal{C}) & & \mathbf{Lens}(\mathcal{C}) \\ \nwarrow & & \nearrow \\ & \mathbf{Adap}(\mathcal{C}) & \end{array} \quad (3.29)$$

Adapters, lenses and colenses are all instances of optics [CEG+20] and recently $\mathbf{Int}(\mathcal{C})$ was also shown to be a theory of optics (this was first noted in [Hed23], and then expanded upon in private communication between Hedges and Milewski). Thus we conjecture this diamond arises from relations between the actions generating the optics [Rom20].

⁷One can see maps $S \rightarrow I$ as spans $S \xlongequal{\quad} S \rightarrow I$, thereby fitting this example into the more general pattern of ‘systems are compositions with a special left boundary’.

The second is to think of these maps as display maps for some kind of type dependency of S over I . From this point of view we are more interested in the fibers of such map: they give us, for any observation $i : I$ at the interface, a set $s : S(i) = \mathbf{observe}^{-1}(i)$ of states compatible with it.

There's many reasons to prefer this second interpretation. The first is given by the way we are going to define the reindexing operations of **Set**⁸, which treat **observe** as a display map. The second is by the way **Set** is used in the context of categorical systems theory, namely as a 'semantical theory': we are going to map other systems theory to behavioural theories to model the act of substantiating a systems' specification with a an actual observable behaviour (see Section 4). The examples then suggest **observe** is indeed a display map.

Thus, let's now look at how $\mathbf{Span}(\mathcal{E})$ indexes observations.

Given an observation $\mathbf{observe}_S : S \rightarrow I$, we can reindex it functorially along a span $I \xleftarrow{p_\ell} X \xrightarrow{p_r} I'$ by pulling back along p_ℓ first and then composing with p_r (this is called the pull-push action of spans):

$$\mathcal{E}(I \xleftarrow{p_\ell} X \xrightarrow{p_r} I') : \mathcal{E}(I) \longrightarrow \mathcal{E}(I')$$

$$\begin{array}{ccc} S & & p_\ell^* S \\ \mathbf{observe}_S \downarrow & \longmapsto & \downarrow p_\ell^* \mathbf{observe}_S \\ I & & X \\ & & \downarrow p_r \\ & & I' \end{array} \quad (3.32)$$

We can visualize this better by arranging an appropriate diagram:

$$\begin{array}{ccccc} S & \longleftarrow & p_\ell^* S & \xlongequal{\quad} & p_\ell^* S \\ \mathbf{observe}_S \downarrow & & \downarrow p_\ell^* \mathbf{observe}_S & & \downarrow \mathbf{Set}(p)(\mathbf{observe}_S) \\ I & \xleftarrow{p_\ell} & X & \xrightarrow{p_r} & I' \end{array} \quad (3.33)$$

Given a map $I \xrightarrow{h} J$, we define a profunctor that maps a pair of observation maps to the set of maps between their domains that make the evident square commute:

$$\mathcal{E}(I)^{\text{op}} \times \mathcal{E}(J) \xrightarrow{\mathcal{E}(h)} \mathbf{Set}$$

$$\begin{array}{ccc} \begin{array}{c} S \\ \mathbf{observe}_S \downarrow \\ I \end{array} & \begin{array}{c} R \\ \mathbf{observe}_R \downarrow \\ J \end{array} & \longmapsto \left\{ \begin{array}{ccc} S & \dashrightarrow & R \\ \mathbf{observe}_S \downarrow & & \downarrow \mathbf{observe}_R \\ I & \xrightarrow{h} & J \end{array} \right\} \end{array} \quad (3.34)$$

⁸This might seem circular but the definition of **Set** is actually mathematically motivated. So we use the mathematics as a suggestion of which interpretation fits it better, as one should do.

Finally, we have a map on squares:

$$\begin{array}{ccc}
 \begin{array}{ccc}
 I & \xrightarrow{h} & J \\
 p_\ell \uparrow & & \uparrow q_\ell \\
 X & \xrightarrow{\sigma} & Y \\
 p_r \downarrow & & \downarrow q_r \\
 I' & \xrightarrow{k} & J'
 \end{array} & \mapsto & \begin{array}{ccc}
 \mathcal{E}(I) & \xrightarrow{\mathcal{E}(p)} & \mathcal{E}(J) \\
 \mathcal{E}(h) \downarrow & \xRightarrow{\mathcal{E}(\sigma)} & \downarrow \mathcal{E}(k) \\
 \mathcal{E}(I') & \xrightarrow{\mathcal{E}(q)} & \mathcal{E}(J')
 \end{array}
 \end{array} \quad (3.35)$$

given, again, by stacking:

$$\begin{array}{ccc}
 \mathcal{E}(h)(S, R) & \xrightarrow{\mathcal{E}(\sigma)_{S, R}} & \mathcal{E}(k)(\mathcal{E}(f)(S), \mathcal{E}(g)(R)) \\
 \\
 \begin{array}{ccc}
 S & \xrightarrow{\quad} & R \\
 \text{observes} \downarrow & \text{observer}_R \downarrow & \\
 I & \xrightarrow{h} & J
 \end{array} & \mapsto & \begin{array}{ccc}
 S & \xrightarrow{\quad} & R \\
 \downarrow & & \downarrow \\
 I & \xrightarrow{h} & J \\
 \uparrow & & \uparrow \\
 X & \xrightarrow{\sigma} & Y \\
 \downarrow & & \downarrow \\
 I' & \xrightarrow{k} & J'
 \end{array}
 \end{array} \quad (3.36)$$

Thus we have a systems theory:

$$\mathcal{E} : \mathbb{S}\mathbf{pan}(\mathcal{E})^\top \xrightarrow{\text{unitary lax}} \mathbb{C}\mathbf{at} \quad (3.37)$$

In particular, when $\mathcal{E} = \mathbf{Set}$, then $\mathbf{Set} := \mathbb{S}\mathbf{pan}(\mathbf{Set})$ and we define:

$$\mathbf{Set} := \mathbf{Set}(\mathbf{Set}) : \mathbf{Set}^\top \xrightarrow{\text{unitary lax}} \mathbb{C}\mathbf{at}. \quad (3.38)$$

Notably, in this case, $I \mapsto \mathbf{Set}/I \cong \mathbf{Set}^I$ (where the latter sends a ‘display map’ to the indexed set of its fibers).

Example 3.10 (Generalized behavioural theories). Following Example 2.5, if we start from a cartesian strong display map category $(\mathcal{E}, \mathcal{D})$ instead, we still get a theory over $\mathbb{S}\mathbf{pan}(\mathcal{E}, \mathcal{D})$ mapping an object I to the display maps over it $\mathcal{D}/I$ (the *display slice* over I). The same definitions as above work with this restriction since, by assumption, display maps can be both pulled back along arbitrary maps and composed with each other.

In fact, one can cook up a behavioural theory for any *comprehension category with strong sums* [Jac99, Definition 10.3.3], these being fibrations $p : \mathcal{P} \rightarrow \mathcal{B}$ with extra structure and properties. In particular these induce a strong display map category structure on $\mathcal{B}$, which allows to define $\mathbb{S}\mathbf{pan}(\mathcal{B}, \mathcal{D})$, and this double category reindexes the fibers of p instead of display slices. We leave details to the interested reader.

Example 3.11 (Structured cospans).

Matteo: TODO

3.1. Doctrines of systems. A doctrine of systems is a uniform way to specify theories of systems given some data. Most of the theories we described above are actually doctrines, since we defined them parametric on some data:

- (1) a behavioural theory is defined for each Cartesian category $\mathcal{C}$,
- (2) a theory of Moore machines is defined for every fibration $\pi : \mathcal{E} \rightarrow \mathcal{C}$ with a section T ,
- (3) a theory of coalgebras is defined for every category $\mathcal{C}$ with pullbacks,

and so on.

Definition 3.3. A **doctrine of systems** is a 2-functor into $\mathbf{Th}$.

The 2-category $\mathbf{Th}$ has system theories as objects and the following as 1-cells:

Definition 3.4. A **map of system theories** is a pair $(F, F^b) : \mathbf{Sys}_1 \rightarrow \mathbf{Sys}_2$ where F is a lax double functor while F^b is a vertical lax-natural transformation.

$$\begin{array}{ccc}
 \mathbb{C}_1^\top & \xrightarrow{\text{Sys}_1} & \mathbf{Cat} \\
 F^\top \downarrow & F^b \Downarrow & \\
 \mathbb{C}_2^\top & \xrightarrow{\text{Sys}_2} &
 \end{array} \tag{3.39}$$

The 2-cells in $\mathbf{Th}$ are pairs of an horizontal natural transformation and a modification [Gra19].

4. BEHAVIOURS

Behaviours in CST are simply maps $\mathbf{B} : \mathbf{Sys} \rightarrow \mathcal{C}$ into a behavioural theory. By default, we consider behaviours valued in $\mathbf{Set}(\mathbf{Set})$, but one can consider ‘structured behaviours’ into $\mathcal{C}$ when one wants to keep track of extra structure on the set of behaviours of a system.

The intuition behind this definition of behaviours the following definition is the following. First of all, B maps interfaces to objects of observations we can make about them. Every theory of behaviour gives its own notion of what ‘observation’ means: it could be mere points, it could be distributions, it could be sequences of observations, and so on (examples will prove the variety this definition allows). Then, we map every system to an object of states (which are *observations we can make about the ways the system can be*) and a map that displays the dependency of said state on the observations on its interface (thus *observations we can make about the ways the system can change*).

But mapping interfaces and systems to observations it’s only half of what a behaviour does. The rest of the data (called below vertical and horizontal naturality) witness the *compositionality* of such assignment. We say how maps of systems translate to maps between their behaviours and, most importantly, we say how composing systems and composing behaviours interact. The latter is thus a direct witness of *emergence* of behaviour.

Definition 4.1. A **theory of behaviour** or just **behaviour** for a systems theory **Sys** is a map of system theories $B : \mathbb{C} \rightarrow \mathcal{E}$:

$$\begin{array}{ccc} \mathbb{C}^\top & \xrightarrow{\text{Sys}} & \mathbb{Cat} \\ B^\top \downarrow & B^\flat \Downarrow & \\ \mathbb{Span}(\mathcal{E})^\top & \xrightarrow{\mathcal{E}/-} & \end{array} \quad (4.1)$$

Hence B is given by

- (1) a unitary lax double functor

$$B : \mathbb{C} \xrightarrow{\text{unitary lax}} \mathbb{Span}(\mathcal{E}) \quad (4.2)$$

- (2) a vertical lax-natural transformation whose components are given, for every $I : \mathbb{C}$, by

$$B_I^\flat : \text{Sys}(I) \longrightarrow \mathcal{E}/B(I). \quad (4.3)$$

- (3) a laxator witnessing vertical naturality for every process $p : I \multimap J$ in $\mathbb{C}$:

$$\begin{array}{ccc} \text{Sys}(I) & \xrightarrow{B_I^\flat} & \mathcal{E}/B(I) \\ \text{Sys}(p) \downarrow & B_p^\flat \nearrow & \downarrow \mathcal{E}/B(p) \\ \text{Sys}(J) & \xrightarrow{B_J^\flat} & \mathcal{E}/B(J) \end{array} \quad \text{or alternatively:} \quad \begin{array}{ccc} \text{Sys}(I) & \xlongequal{\quad} & \text{Sys}(I) \\ \text{Sys}(p) \downarrow & B_p^\flat \Downarrow & \downarrow B_I^\flat \\ \text{Sys}(J) & & \mathcal{E}/I \\ B_J^\flat \downarrow & \Downarrow & \downarrow \mathcal{E}/B(p) \\ \mathcal{E}/J & \xlongequal{\quad} & \mathcal{E}/J \end{array} \quad (4.4)$$

which (when p represents a coupling of some sort, hence a composition of systems) sends a behaviour of the composite to a composite of behaviours,

- (4) a square in $\mathbb{Cat}$ witnessing horizontal naturality for every map of interfaces $h : I \rightarrow J$ in $\mathbb{C}$:

$$\begin{array}{ccc} \text{Sys}(I) & \xrightarrow{B_I^\flat} & \mathcal{E}/B(I) \\ \text{Sys}(h) \downarrow & \xlongequal{B_h^\flat} & \downarrow \mathcal{E}/B(h) \\ \text{Sys}(J) & \xrightarrow{B_J^\flat} & \mathcal{E}/B(J) \end{array} \quad (4.5)$$

which sends every map of systems (over h) to a map of their behaviours (over $B(h)$)

This data satisfies the following coherences:

- (1) The laxators witnessing vertical functoriality compose; hence for each composable pair of processes $I \xrightarrow{p} J \xrightarrow{q} K$ in $\mathbb{C}$, we have:

$$\begin{array}{ccc} \text{Sys}(I) & \xrightarrow{B_I^\flat} & \mathcal{E}/B(I) \\ \text{Sys}(p) \downarrow & B_p^\flat \nearrow & \downarrow \mathcal{E}/B(p) \\ \text{Sys}(J) & \xrightarrow{B_J^\flat} & \mathcal{E}/B(J) \\ \text{Sys}(q) \downarrow & B_q^\flat \nearrow & \downarrow \mathcal{E}/B(q) \\ \text{Sys}(K) & \xrightarrow{B_K^\flat} & \mathcal{E}/B(K) \end{array} = \begin{array}{ccc} \text{Sys}(I) & \xrightarrow{B_I^\flat} & \mathcal{E}/B(I) \\ \text{Sys}(p) \downarrow & & \downarrow \mathcal{E}/B(p) \\ \text{Sys}(J) & & \mathcal{E}/B(J) \\ \text{Sys}(q) \downarrow & B_{p \circ q}^\flat \nearrow & \downarrow \mathcal{E}/B(q) \\ \text{Sys}(K) & \xrightarrow{B_K^\flat} & \mathcal{E}/B(K) \end{array} \quad (4.6)$$

Moreover, $B(1) = 1$.

- (2) The squares witnessing horizontal naturality commute with the laxators of **Sys** and $B^*\mathcal{E}/-$ (the composite of B and $\mathcal{E}/-$); hence for each composable pair of maps of interfaces $I \xrightarrow{h} J \xrightarrow{k} K$ in $\mathbb{C}$, we have:

$$\begin{array}{ccccc}
 \mathbf{Sys}(I) & \xlongequal{\quad} & \mathbf{Sys}(I) & \xrightarrow{B_I^\flat} & \mathcal{E}/B(I) & \xlongequal{\quad} & \mathcal{E}/B(I) \\
 \downarrow \mathbf{Sys}(h \circ k) & & \downarrow \mathbf{Sys}(h) & \xlongequal{B_h^\flat} & \downarrow \mathcal{E}/B(h) & & \downarrow \mathcal{E}/(B(h \circ k)) \\
 \downarrow = \ell_{h,k}^{\mathbf{Sys}} \Rightarrow & \mathbf{Sys}(J) & \xrightarrow{B_J^\flat} & \mathcal{E}/B(J) & = & \mathbf{Sys}(J) & \xrightarrow{B_J^\flat} & \mathcal{E}/B(J) & = \ell_{h,k}^{B^*\mathcal{E}/-} \Rightarrow & \downarrow \\
 \downarrow & & \downarrow \mathbf{Sys}(k) & \xlongequal{B_k^\flat} & \downarrow \mathcal{E}/B(k) & & \downarrow \mathbf{Sys}(k) & \xlongequal{B_k^\flat} & \downarrow \mathcal{E}/B(k) & \\
 \mathbf{Sys}(K) & \xlongequal{\quad} & \mathbf{Sys}(K) & \xrightarrow{B_K^\flat} & \mathcal{E}/B(K) & \xlongequal{\quad} & \mathbf{Sys}(K) & \xrightarrow{B_K^\flat} & \mathcal{E}/B(K) & \xlongequal{\quad} & \mathcal{E}/B(K)
 \end{array} \tag{4.7}$$

Notice the laxator of $B^*\mathcal{E}/-$ is given by

$$\mathcal{E}/B(h) \circ \mathcal{E}/B(k) \xrightarrow{\ell_{B(h), B(k)}^{\mathcal{E}/-}} \mathcal{E}/(B(h) \circ B(k)) \xrightarrow{\mathcal{E}/\ell_{h,k}^B} \mathcal{E}/B(h \circ k). \tag{4.8}$$

Moreover, $B(1) = 1$.

- (3) For every square $p \bullet \xrightarrow{h} J$ in $\mathbb{C}$, the following ‘cube’ commutes:
- $$\begin{array}{ccc}
 I & \xrightarrow{h} & J \\
 p \bullet \downarrow \alpha \Rightarrow \bullet p' & & \bullet p' \\
 I' & \xrightarrow{h'} & J'
 \end{array}$$

$$\begin{array}{ccccccc}
 \mathbf{Sys}(I) & \xrightarrow{\mathbf{Sys}(h)} & \mathbf{Sys}(J) & \xlongequal{\quad} & \mathbf{Sys}(J) & & \mathbf{Sys}(I) & \xlongequal{\quad} & \mathbf{Sys}(I) & \xrightarrow{\mathbf{Sys}(h)} & \mathbf{Sys}(J) \\
 \downarrow \mathbf{Sys}(p) & \parallel \mathbf{Sys}(\alpha) & \downarrow \mathbf{Sys}(p') & \parallel & \downarrow B_J^\flat & & \downarrow \mathbf{Sys}(p) & \parallel & \downarrow B_I^\flat & \parallel B_h^\flat & \downarrow B_J^\flat \\
 \mathbf{Sys}(I') & \xrightarrow{\mathbf{Sys}(h')} & \mathbf{Sys}(J') & \xlongequal{B_{p'}^\flat} & \mathcal{E}/B(J) & = & \mathbf{Sys}(I') & \xlongequal{B_p^\flat} & \mathcal{E}/B(I) & \xrightarrow{\mathcal{E}/B(h)} & \mathcal{E}/B(J) \\
 \downarrow B_{I'}^\flat & \parallel B_{h'}^\flat & \downarrow B_{J'}^\flat & \parallel & \downarrow \mathcal{E}/B(p') & & \downarrow B_{I'}^\flat & \parallel & \downarrow \mathcal{E}/B(p) & \parallel \mathcal{E}/B(\alpha) & \downarrow \mathcal{E}/B(p') \\
 \mathcal{E}/B(I') & \xrightarrow{\mathcal{E}/B(h')} & \mathcal{E}/B(J') & \xlongequal{\quad} & \mathcal{E}/B(J') & & \mathcal{E}/B(I') & \xlongequal{\quad} & \mathcal{E}/B(I') & \xrightarrow{\mathcal{E}/B(h')} & \mathcal{E}/B(J')
 \end{array} \tag{4.9}$$

Notice we used the ‘transposed’ version of the vertical laxators of $B^\flat$ (diagram on the right of (4.4)).

Example 4.1 (States). One of the simplest notion of behaviour for a system is given by the idea of ‘state space’. This is the very classical stance that what can be observed about a system is its permanence in a state which varies in a prescribed space.

Clearly such a theory of behaviour can be defined for Moore machines. Let $\mathbf{Moore}_{(F,T)} : \mathbb{Lens}^\top \xrightarrow{\text{unitary lax}} \mathbf{Cat}$ be a theory of Moore machines, then there is again a theory of behaviour

$$\begin{array}{ccc}
 \mathbb{Lens}^\top & \xrightarrow{\mathbf{Moore}_{(F,T)}} & \mathbf{Cat} \\
 1^\top \downarrow \text{states} \Downarrow & & \uparrow c/- \\
 \mathbf{Span}(\mathcal{C})^\top & &
 \end{array} \tag{4.10}$$

which is trivial on the interfaces and maps Moore machines to their state spaces:

$$\begin{aligned} \mathbf{states}_{\begin{pmatrix} I \\ O \end{pmatrix}} : \mathbf{Moore}_{(F,T)}\left(\begin{pmatrix} I \\ O \end{pmatrix}\right) &\longrightarrow \mathcal{C} \\ (S, \begin{pmatrix} \text{update} \\ \text{expose} \end{pmatrix}) : \begin{pmatrix} TS \\ S \end{pmatrix} \rightleftharpoons \begin{pmatrix} I \\ O \end{pmatrix}) &\longmapsto S. \end{aligned} \quad (4.11)$$

Example 4.2 (Paths). Labelled transition systems admit a behaviour $\mathbf{P} : \mathbf{Trans} \rightarrow \mathbf{Set}$ given by paths:

$$\begin{array}{ccc} \mathbf{Alph}^\top & \xrightarrow{\mathbf{Trans}} & \mathbf{Cat} \\ (-)^*\top \downarrow & P \Downarrow & \uparrow \mathbf{Set}/- \\ \mathbf{Span}(\mathbf{Set})^\top & \xrightarrow{\mathbf{Set}/-} & \mathbf{Cat} \end{array} \quad (4.12)$$

where

- (1) $\mathbf{Trans} : \mathbf{Alph}^\top \xrightarrow{\text{unitary lax}} \mathbf{Cat}$ is the theory of systems introduced in Example 3.4;
- (2) $(-)^* : \mathbf{Alph} \rightarrow \mathbf{Span}(\mathbf{Set})$ sends a finite alphabet Σ to the free monoid Σ^* , an alphabet mapping $h : \Sigma \rightarrow \Sigma'$ to the morphism of monoids $h^* : \Sigma^* \rightarrow \Sigma'^*$, an alphabet reduction $p : \Sigma \leftarrow \Xi$ to the span $\Sigma^* \xleftarrow{p^*} \Xi^* = \Xi^*$, and a commutative square to an obvious morphism of spans;
- (3) The component at $\Sigma : \mathbf{Alph}$ of P is

$$\begin{aligned} P_\Sigma : \mathbf{Trans}(\Sigma) &\longrightarrow \mathbf{Set}/\Sigma^* \\ S \times \Sigma \xrightarrow{\delta} S &\longmapsto \{s \xrightarrow{w}_\delta s', w \in \Sigma^*\} \xrightarrow{\pi_{\Sigma^*}} \Sigma^* \end{aligned} \quad (4.13)$$

where $s \xrightarrow{w}_\delta s'$ denotes a triple $(w, s, s') \in \Sigma^* \times S \times S$ such that s' can be reached from s by following the transitions $w_1 \cdots w_n = w$ determined by the transition map δ .

- (4) The vertical lax natural structure of P is given, for each alphabet reduction $p : \Sigma \leftarrow \Xi$, by a natural transformation P_p filling the square:

$$\begin{array}{ccc} \mathbf{Trans}(\Sigma) & \xrightarrow{P_\Sigma} & \mathbf{Set}/\Sigma \\ \mathbf{Trans}(p) \downarrow & P_p \nearrow & \downarrow \mathbf{Set}/p^* \\ \mathbf{Trans}(\Xi) & \xrightarrow{P_\Xi} & \mathbf{Set}/\Xi \end{array} \quad (4.14)$$

We define it as follows. Given $S \times \Sigma \xrightarrow{\delta} S : \mathbf{Trans}(\Sigma)$, one gets, following the bottom path:

$$\begin{aligned} S \times \Sigma \xrightarrow{\delta} S &\mapsto S \times \Xi \xrightarrow{S \times p} S \times \Sigma \xrightarrow{\delta} S \mapsto \{s \xrightarrow{v}_{\mathbf{Trans}(p)(\delta)} s', v \in \Xi^*\} \\ &\downarrow \pi_{\Xi^*} \\ &\Xi^* \end{aligned} \quad (4.15)$$

Following the top path, one gets instead:

$$\begin{aligned} S \times \Sigma \xrightarrow{\delta} S &\mapsto \{s \xrightarrow{w}_\delta s', w \in \Sigma^*\} \xrightarrow{\pi_{\Sigma^*}} \Sigma^* \\ &\mapsto \{s \xrightarrow{w}_\delta s', w = p^*(v), v \in \Xi^*\} \xrightarrow{p^*} \Xi^* \xrightarrow{p^*} \Sigma^* \\ &\quad \downarrow \mathbf{Set}/p^*(\pi_{\Sigma^*}) \quad \downarrow \pi_{\Sigma^*} \end{aligned} \quad (4.16)$$

Thus the components of P_p are:

$$\begin{array}{ccc} \{s \xrightarrow{v} \mathbf{Trans}(p)(\delta) s', v \in \Xi^*\} & \xrightarrow{P_p(\delta)} & \{s \xrightarrow{w} \delta s', w = p^*(v), v \in \Xi^*\} \\ & \searrow & \swarrow \\ & \Xi^* & \end{array} \quad (4.17)$$

simply by setting $P_p(\delta)(s \xrightarrow{v} \mathbf{Trans}(p)(\delta) s') = s \xrightarrow{p(v)} \delta s'$. Clearly this assignment is natural in δ (since maps of labelled transition systems commute with paths) and $P_p \circ P_q = P_{p \circ q}$ (vertical functoriality).

(5) The naturality squares for P are given, for each horizontal $h : \Sigma \rightarrow \Sigma'$ in $\mathbf{Alph}$, by

$$\begin{array}{ccc} \mathbf{Trans}(\Sigma) & \xrightarrow{P_\Sigma} & \mathbf{Set}/\Sigma^* \\ \mathbf{Trans}(h) \downarrow & \xlongequal{P_h} & \downarrow \mathbf{Set}/h^* \\ \mathbf{Trans}(\Sigma') & \xrightarrow{P_{\Sigma'}} & \mathbf{Set}/\Sigma'^* \end{array} \quad (4.18)$$

defined as

$$\begin{array}{ccc} P_\alpha(\delta, \zeta) : \mathbf{Trans}(h)(\delta, \zeta) & \longrightarrow & \mathbf{Set}/k^*(P_\Sigma(\delta), P_{\Sigma'}(\zeta)) \\ S \times \Sigma \xrightarrow{\varphi \times h} T \times \Sigma' & \xrightarrow{\quad} & \{s \xrightarrow{v} \delta s'\} \xrightarrow{P_h(\varphi)} \{s \xrightarrow{w} \zeta s'\} \\ \delta \downarrow & \downarrow \zeta & \downarrow \pi_{\Sigma'}^* \\ S \xrightarrow{\varphi} T & \xrightarrow{\quad} & \Sigma^* \xrightarrow{h^*} \Sigma'^* \end{array} \quad (4.19)$$

where $P_h(s \xrightarrow{v} \delta s') = \varphi(s) \xrightarrow{h^*(v)} \zeta \varphi(s')$. This is well-defined since, by definition, φ preserve transitions. We leave the reader to prove horizontal functoriality.

Remark 4.1.1. We can already see an example of *map* of behaviours. In fact consider the behaviour of states $\mathbf{states} : \mathbf{Trans} \rightarrow \mathbf{Set}$ and the behaviour of paths $\mathbf{P} : \mathbf{Trans} \rightarrow \mathbf{Set}$:

$$\begin{array}{ccc} \mathbf{Alph}^\top & \xrightarrow{\quad \mathbf{Trans} \quad} & \mathbf{Cat} \\ \downarrow 1^\top & \xRightarrow{\varepsilon^\top} & \downarrow (-)^*\top \\ \mathbf{Span}(\mathbf{Set})^\top & \xRightarrow{\varepsilon^\flat} & \mathbf{Set}/- \end{array} \quad (4.20)$$

The horizontal transformation $\varepsilon : 1 \Rightarrow (-)^*$ picks out the empty word on each set of strings. The modification $\varepsilon^\flat$ is a square in $\mathbf{Cat}$:

$$\begin{array}{ccc} \mathbf{Trans}(\Sigma) & \xrightarrow{\mathbf{states}_\Sigma} & \mathbf{Set}/1 \\ \parallel & \xRightarrow{\varepsilon_\Sigma^\flat} & \downarrow \mathbf{Set}/\varepsilon_\Sigma \\ \mathbf{Trans}(\Sigma) & \xrightarrow{P_\Sigma} & \mathbf{Set}/\Sigma^* \end{array} \quad (4.21)$$

where

$$\varepsilon_{\Sigma}^b(\delta, \zeta) : \mathbf{Trans}(\Sigma)(\delta, \zeta) \longrightarrow \mathbf{Set}/\varepsilon_{\Sigma}(\mathbf{states}_{\Sigma}(\delta), P_{\Sigma}(\zeta))$$

$$\begin{array}{ccc} S \times \Sigma & \xrightarrow{\varphi \times \Sigma} & T \times \Sigma \\ \delta \downarrow & & \downarrow \zeta \\ S & \xrightarrow{\varphi} & T \end{array} \longmapsto \begin{array}{ccc} \mathbf{states}_{\Sigma}(\delta) & \xrightarrow{\varepsilon_{\delta, \zeta}^b} & P_{\Sigma}(\zeta) \\ ! \downarrow & & \downarrow \pi_{\Sigma^*} \\ 1 & \xrightarrow{\varepsilon} & \Sigma^* \end{array} \quad (4.22)$$

is given by $\varepsilon_{\delta, \zeta}^b(s) = (\varepsilon, \varphi(s), \varphi(s))$.

Example 4.3 (Fixpoints).

Matteo: TODO

Example 4.4 (Trajectories).

Matteo: TODO

Example 4.5 (Languages). One of the first notions of ‘behaviour of a system’ one encounters in computer science is that of *language of a finite states machines*. These, indeed, give a theory of behaviours for the theory of finite states machine we described in Example 3.5.

Given an FSM $S = (S, \Sigma, \delta, S^*)$, its language is the subset of Σ^* given by those words which, when fed to S , leave it in an accepting state (one in S^*). More precisely, we first have to pick an initial state $s_0 \in S$, then we can start using δ , inputting the given word one character at a time, until we are done. Thus the language of an S is really a family of subsets of Σ^* , given by the recursively-defined map

$$L_S : S \longrightarrow \mathcal{P}\Sigma^*$$

$$s_0 \longmapsto \{\sigma_1 \cdots \sigma_n \mid \sigma_2 \cdots \sigma_n \in L_S(\delta(s_0, \sigma_1))\}. \quad (4.23)$$

Notice this family of subsets can also be defined as a ‘bundle of sets’ (a map we use for its fibers)

$$\Lambda(S) := \{(s_0, w) \mid w \in L_S(s_0)\} \rightharpoonup S \times \Sigma^* \xrightarrow{\pi_{\Sigma^*}} \Sigma^* \in \mathbf{Set}/\Sigma^*. \quad (4.24)$$

Thus we see there is a theory of behaviour

$$\begin{array}{ccc} \mathbf{Alph}^{\top} & \xrightarrow{\mathbf{FSM}} & \mathbf{Cat} \\ (-)^*{}^{\top} \downarrow & \Downarrow \Lambda & \uparrow \mathbf{Set}/- \\ \mathbf{Span}(\mathbf{Set})^{\top} & & \end{array} \quad (4.25)$$

where

- (1) $(-)^* : \mathbf{Alph} \rightarrow \mathbf{Span}(\mathbf{Set})$ is the same as in Example 4.2,
- (2) the component at $\Sigma : \mathbf{Alph}$ of Λ is given by

$$\Lambda_{\Sigma} : \mathbf{FSM}(\Sigma) \longrightarrow \mathbf{Set}/\Sigma^*$$

$$(S, \Sigma, \delta, S^*) \longmapsto \Lambda(S) : \{(s_0, w) \mid w \in L_S(s_0)\} \rightarrow \Sigma^* \quad (4.26)$$

The functoriality properties of these assignments can be proven by making the following observation. For each $n \in \mathbb{N}$ (including 0), there are finite states machines $\mathbf{Fin} \, n = (\mathbf{Fin} \, n +$

$1, \text{Fin } n, _ + 1, \{n\}$). Explicitly, these have $n + 1$ states and are defined over an alphabet with n symbols. The transition map is

$$\begin{aligned} _ + 1 : \text{Fin } n + 1 \times \text{Fin } n &\longrightarrow \text{Fin } n + 1 \\ (k, i) &\longmapsto k + 1 \end{aligned} \quad (4.27)$$

and the only accepting state is $n \in \text{Fin } n + 1$.

A map from this machine to another like $\mathbf{S}$, mediated by a map of alphabets $\sigma : \text{Fin } n \rightarrow \Sigma$, looks like this:

$$\begin{array}{ccc} \text{Fin } n + 1 \times \text{Fin } n & \xrightarrow{s \times \sigma} & S \times \Sigma \\ \downarrow \scriptstyle _ + 1 & & \downarrow \scriptstyle \delta \\ \text{Fin } n + 1 & \xrightarrow{s} & S \\ \cup \scriptstyle \{n\} & \subseteq & \cup \scriptstyle S^* \end{array} \quad (4.28)$$

Concretely, this is a map $s : \text{Fin } n + 1 \rightarrow S$ selecting a sequence of $n + 1$ states $s_0, \dots, s_n$ such that

$$\forall 0 \leq k \leq n - 1, s_{k+1} = \delta(s_k, \sigma_k) \quad \text{and} \quad s_n \in S^*. \quad (4.29)$$

In other words, this data amounts to a word $\sigma_1 \cdots \sigma_n \in \Sigma^*$ and a state $s_0 \in S$ such that $\mathbf{S}$ accepts $\sigma_1 \cdots \sigma_n$ in n steps when starting from s_0 .

Notice this also works for the empty word: a map from $\text{Fin } 0$ is given by just a choice of state $s_0 \in S^*$, meaning $\mathbf{S}$ accepts the empty word when starting from s_0 .

Therefore we see $((-)^*, \Lambda)$ is a sum of corepresentable behaviours, namely $\sum_{n \geq 0} \mathbf{FSM}(\text{Fin } n, -)$. Observe the symmetry restored: $(-)^*$ is also the functor $\sum_{n \geq 0} \mathbf{Set}(\text{Fin } n, -)!$

Remark 4.1.2. Notice the observation that (4.26) is a sum of corepresentables was already valid for the behaviour of paths $\mathbf{P}$ in Example 4.2. The systems corepresenting the behaviours are the same as the ones just described except for the acceptance predicate, which is not there for labelled transition systems.

In this light, the map described in Remark 4.1.1 is simply the inclusion of **states** = $\mathbf{Trans}(\text{Fin } 0, -)$ into the sum $\mathbf{P} = \sum_{n \geq 0} \mathbf{Trans}(\text{Fin } n, -)$.

4.1. Generalized behaviours. Generalized behaviours are those landing in a generalized behavioural theory (Example 3.10).

Example 4.6 (Reachability). In Example 4.2 we've seen finite-length paths are a theory of behaviour for labelled transition systems. If one were to care just about reachability between states though, paths would be overkill: they remember, yes, whether a certain state is reachable from another one, but also all possible way in which this can happen. This can be a plus in some settings, but a burden in others.

To forget about paths and only keep the reachability relation we have to truncate the information we have about the type of paths between two states to just a proposition. In other words, we want to turn our dependent types to predicates. Doing so will yield a non-standard behaviour of reachability.

Truncation can be expressed as a morphism of system theories $\| - \|_0 : \mathbf{Set} \rightarrow \mathbf{Set}_0$, where the latter is a theory like $\mathbf{Set}$ except on objects, where $\mathbf{Set}/-$ is replaced by $\text{Sub}(-)$. Hence systems are not display maps anymore, but just subobjects.

The natural transformation $\text{im} : \mathbf{Set}/- \rightarrow \text{Sub}(-)$, which sends a display map $\pi : S \rightarrow A$ to its image $\text{im } \pi \subseteq A$, induces the desired $\| - \|_0$.

Now reachability is the non-standard behaviour $\mathbf{R} : \mathbf{Trans} \rightarrow \mathbf{Set}_0$ obtained as $\|\mathbf{P}\|_0$, where $\mathbf{P}$ is the behaviour of paths from Example 4.2.

4.2. Doctrines of behaviour. Like theories of systems, theories of behaviours can be gathered in uniform *doctrines* which functorially specify a way to look at theories of a doctrine of systems:

Definition 4.2. A **doctrine of behaviour** for the doctrine $\mathbb{D}\text{octrine}$ is map of doctrines of systems into the behavioural doctrine $\mathbf{Set}$:

$$\begin{array}{ccc}
 \mathbb{D}\text{ata} & \xrightarrow{\text{Doctrine}} & \mathbf{Th} \\
 \mathfrak{B}^\top \downarrow & \mathfrak{B}^b \Downarrow & \uparrow \mathfrak{B}^h \\
 \mathbb{C}\text{artCat} & &
 \end{array} \quad (4.30)$$

Remark 4.2.1. The laxity of B^b relates the behaviours of the parts of a system to the behaviour of the whole system. The non-invertibility of such a map witnesses emergent behaviours. In [Mye22, Theorem 5.3.3.1], Myers proves that a large class of behaviours for the doctrine of Moore machines does not, in fact, exhibit emergence, by showing such the laxity of B^b is invertible.

Example 4.7. A large class of behaviours are corepresentables, i.e. defined by simulations of an archetypal system exhibiting that behaviour. Thus there is a **doctrine of corepresentable behaviour** on the doctrine of pointed theories:

$$\begin{array}{ccc}
 \mathbf{Th}_* & \xrightarrow{\mathfrak{u}} & \mathbf{Th} \\
 \mathbf{Set} \downarrow & \text{Hom}^h \Downarrow & \uparrow \mathfrak{B}^h \\
 \mathbb{C}\text{artCat} & &
 \end{array} \quad (4.31)$$

An object of $\mathbf{Th}_*$ is a systems theory equipped with a distinguished system (hence a pair of an interface and a system over it) which is used as archetype for a certain kind of behaviour.

The transformation Hom^h at a pointed theory $(\mathbf{Sys} : \mathbb{C}^\top \rightarrow \mathbb{C}\text{at}, \mathbf{B} : \mathbf{Sys}(I))$ is the map of system theories $\mathbb{C}^h(\mathbf{B}, -) : \mathbf{Sys} \rightarrow \mathbf{Set}(\mathbf{Set})$ defined as follows. Its two components are the horizontal hom-functor $\mathbb{C}^h(I, -) : \mathbb{C} \rightarrow \mathbf{Set}$ and the similar fiberwise hom-functor

$$\mathbb{C}^h(\mathbf{B}, -)^b : \mathbf{Sys} \Rightarrow \mathbf{Set}/\mathbb{C}^h(I, -). \quad (4.32)$$

This latter functor sends a system $\mathbf{S} : \mathbf{Sys}(J)$ to the $\mathbb{C}^h(I, J)$ -indexed family of sets sending a map of interfaces $k : I \rightarrow J$ to the set $\mathbf{Sys}(k)(\mathbf{B}, \mathbf{S})$ of maps of systems mediated by k .

Example 4.8. Let $\mathbf{MooreData}^!$ be the 2-category of fibrations of categories with a terminal object and a section thereof. These amount to a fibration $p : \mathcal{E} \rightarrow \mathcal{C}$ such that $p(1_{\mathcal{E}}) = 1_{\mathcal{C}}$, and a section $T : \mathcal{C} \rightarrow \mathcal{E}$ such that $T(1_{\mathcal{C}}) = 1_{\mathcal{E}}$. In this situation, one can build the Moore machine

$\text{fix} : \begin{pmatrix} T^1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ which ‘does nothing’. Hence we get a *doctrine of fixpoints* by using such a machine as the archetype for the behaviour of a still system:

$$\begin{array}{ccc}
 \text{MooreData}^! & & \\
 \downarrow (\text{Moore}, \text{fix}:\text{Moore}(1)) & \searrow \text{Moore} & \\
 \text{Th}_* & \xrightarrow{\mathcal{U}} & \text{Th} \\
 \downarrow \text{Set} & \text{Hom}^h \Downarrow & \\
 \text{CartCat} & \xrightarrow{\text{Beh}} & \text{Th}
 \end{array} \tag{4.33}$$

5. TIME

We can consider $\mathbf{Sys} : \mathbb{C}^\top \xrightarrow{\text{unitary lax}} \mathbf{Cat}$ a kind of category, by leaving the indexing on interfaces implicit. Its objects $S \in \mathbf{Sys}$ are systems $S \in \mathbf{Sys}(A)$ over a certain (left implicit) interface $A \in \mathbb{C}$, and whose hom-sets $\mathbf{Sys}(S, T)$ are given by maps of systems $\varphi \in \mathbf{Sys}(h)(S, T)$, as h varies among maps of interfaces.

Formally, this corresponds to a flavour of Grothendieck construction of $\mathbf{Sys}$, in the form of a double category whose objects are pairs $(A \in \mathbb{C}, S \in \mathbf{Sys}(A))$, denoted as $\begin{pmatrix} S \\ A \end{pmatrix}$, whose tight 1-cells are maps of systems and interfaces, and whose loose 1-cells $\begin{pmatrix} S \\ A \end{pmatrix} \rightarrow \begin{pmatrix} T \\ B \end{pmatrix}$ are compositions $p : A \rightarrow B$ such that $S \cdot p = T$.

We denote this double category as $\mathbf{Sys}$. We make good use of the tight part of this double category. In there, we can define universal constructions of systems.

Definition 5.1. A **diagram of systems** is a category $\mathcal{J}$ together with a functor $D : \mathcal{J} \rightarrow \mathbf{Sys}$ picking out a system D_j with interface I_j for each $j \in \mathcal{J}$ and a map of systems $D_f : D_j \rightarrow D_i$ over the map of interfaces $I_j \rightarrow I_i$ for each 1-cell $f : j \rightarrow i \in \mathcal{J}$.

Definition 5.2. The **limit** (resp. **colimit**) of a diagram of systems is the categorical limit (resp. colimit) of the diagram in the tight category of $\mathbf{Sys}$.

Remark 5.2.1. A category like $\mathcal{J}$ can be turned into a double category by equipping it with trivial loose morphisms only, and then into a trivial theory of systems by assigning to each $j \in \mathcal{J}$ the terminal category 1. Then a diagram $D : \mathcal{J} \rightarrow \mathbf{Sys}$ is a (necessarily taut) map of theories.

The following generalizes Example 4.7:

Proposition 5.3. For each diagram of systems $T : \text{Time} \rightarrow \mathbf{Sys}$, there is a **jointly corepresentable behaviour** $\mathbf{Sys}(T, -)$ given on each $A \in \mathbb{C}$ by the functor

$$\mathbf{Sys}(T, -) : \mathbf{Sys}(A) \longrightarrow \mathbf{Set} / \text{colim}_{t \in \text{Time}} \mathbb{C}(I_t, A) \tag{5.1}$$

defined as $\mathbf{Sys}(T, X) = \text{colim}_{t \in \text{Time}} \left(\sum_{h \in \mathbb{C}(I_t, A)} \mathbf{Sys}(h)(T_t, X) \xrightarrow{\text{fst}} \mathbb{C}(I_t, A) \right)$.

The idea here is to embody a topology of time in a category and specifically in the functor picking out ‘walking trajectories’ T_t for every ‘shape of time’ t . This allows to be very flexible and precise in our meaning of time. For instance, here’s three different ways to talk about discrete, finite-length trajectories of Moore machines:

Example 5.1. Consider the theory $\mathbf{Moore}_P$ of non-deterministic Moore machines in $\mathbf{Set}$. Let $\mathbf{Time} = (\mathbb{N}, \leq)$ and define $T : \mathbf{Time} \rightarrow \mathbf{Moore}_P$ as follows. The non-deterministic Moore machine T_n has interface⁹ $\begin{pmatrix} 1 \\ n \end{pmatrix}$ and dynamics representable as follows:

$$\begin{array}{ccccccc} 1 & \longrightarrow & 2 & \longrightarrow & \dots & \longrightarrow & n \\ \bullet & & \bullet & & & & \bullet \end{array} \quad (5.2)$$

An inequality $n \leq m$ is then mapped to the map of systems $T_n \rightarrow T_m$ that maps the states of the first to the first n states of the latter.

Example 5.2. Instead of $(\mathbb{N}, \leq)$ let $\mathbf{Time}$ be the category whose objects are still $\mathbb{N}$ but whose morphisms $n \rightarrow m$ ($n \leq m$) are given by choices of an offset $h \in \mathbb{N}$ such that $h + n \leq m$. The diagram T is defined as before on object, but now a map $h : n \rightarrow m$ is sent to the function sending $i \in n$ to $i + h \in m$.

Example 5.3. Consider $B\mathbb{N}$, the one-object category corresponding to the monoid $(\mathbb{N}, 0, +)$. We can send the only object $*$ $\in B\mathbb{N}$ to the Moore machine with states $\mathbb{N}$ and interface $\begin{pmatrix} 1 \\ \mathbb{N} \end{pmatrix}$, with dynamics given by shift:

$$\begin{array}{ccccccc} 1 & \longrightarrow & 2 & \longrightarrow & \dots & \longrightarrow & n & \longrightarrow & \dots \\ \bullet & & \bullet & & & & \bullet & & \end{array} \quad (5.3)$$

On morphisms, the functor is determined by its action on $1 : * \rightarrow *$, which is mapped to the endomorphism $T_* \rightarrow T_*$ given on states by $n \mapsto n + 1$.

Example 5.4. Let $\mathbf{ODE}$ be the theory of differential Moore machines. Consider the category of open intervals of the order $(\mathbb{R}, \leq)$, with morphisms given by length-preserving inclusions. Explicitly, a map $(a, b) \rightarrow (c, d)$ is a real number h such that $c \leq a + h \leq b + h \leq d$. An interval (a, b) can be mapped to the open ODE whose dynamics is $d/dt = 1$ and interface $\begin{pmatrix} 1 \\ (a, b) \end{pmatrix}$. This assignment extends to morphisms since length-preserving maps preserve such constant vector fields.

Example 5.5. Similarly as above, we can instead consider two other notions of time for $\mathbf{ODE}$: restrict ourselves to the intervals $(0, b)$ and their inclusions $(0, b) \rightarrow (0, d)$, or considering the monoid $B(\mathbb{R}, 0, +)$ mapped to the real line with $d/dt = 1$ as dynamics, and sending $\ell : * \rightarrow *$ to the shift map $_ + \ell : \mathbb{R} \rightarrow \mathbb{R}$.

Example 5.6. Let $\mathbf{Petri}$ be the theory of Petri nets, following [Koc22]. As shown in *ibid.*, the category of DAGs and morphisms between them embeds in the category of Petri nets: given a DAG, make its edges places and its nodes transitions.

Proposition 5.4. *Every jointly corepresentable behaviour factors as*

$$\begin{array}{ccc} \mathbf{Sys} & \xrightarrow{\mathbf{Sys}(T, -)} & \mathbf{Set} \\ & \searrow \mathbf{Sys}(T(-), -) & \nearrow \text{colim}_{t \in \mathbf{Time}} \\ & \mathbf{Psh}(\mathbf{Time}) & \end{array} \quad (5.4)$$

⁹We abuse notation by denoting with n also the finite set $\{1, \dots, n\} \subseteq \mathbb{N}$

Proof. Explicitly, the map $\mathbf{Sys}(\mathbb{T}_{(-)}, -)$ is defined on a system $\mathbf{X}$ as

$$\begin{aligned} \mathbf{Sys}(\mathbb{T}_{(-)}, \mathbf{X}) : \mathbf{Time} &\longrightarrow \mathbf{Set} \\ t &\mapsto \mathbf{Sys}(\mathbb{T}_t, \mathbf{X}), \end{aligned} \tag{5.5}$$

making the claimed commutativity evident. $\square$

5.1. Locality.

5.2. Open maps.

6. RELATION WITH COALGEBRAIC SYSTEMS THEORY

In the last decades coalgebra has been hailed as a universal theory of systems [Rut00]. In this section we remark on how the present framework integrates the coalgebraic one.

Let us start by recalling the setup of coalgebraic systems theory. There, systems are coalgebras of an endofunctor $F : \mathcal{S} \rightarrow \mathcal{S}$ where $\mathcal{S}$ is often $\mathbf{Set}$ but can be taken to be any category of choice. A coalgebra $X \xrightarrow{\delta} FX$ is an ‘effectful dynamical system’, with F playing both the role of interface, (e.g. Moore machines with interface $\begin{pmatrix} I \\ O \end{pmatrix}$ are coalgebras of $O \times (-)^I$) and of effect theory (e.g. non-deterministic Moore machines with interface $\begin{pmatrix} I \\ O \end{pmatrix}$ are coalgebras of $O \times P(-)^I$).

Coalgebraic systems theory takes then mainly place in the category $\mathbf{Coalg}(F)$, or related ones. Thus systems are routinely compared by morphisms, and this makes the theory particularly structural. However, composition of systems is completely absent (but this doesn’t mean compositionality is ignored, see e.g. [TP97]).

Matteo: ...

Example 6.1. We can think of a coalgebra $A \rightarrow FA$ as a system with states A and interface F . Natural transformations $\alpha : F \Rightarrow F'$ are ‘lenses’ and one gets an indexed category

$$\mathbf{Coalg} : \mathbf{End}(\mathcal{C}) \rightarrow \mathbf{Cat} \tag{6.1}$$

If $\mathcal{C}$ is additionally finitely complete, we can go further and add another dimension. In fact, in this case, $\mathbf{End}(\mathcal{C})$ is fibred over $\mathcal{C}$ by evaluation at the terminal object:

$$-(1) : \mathbf{End}(\mathcal{C}) \rightarrow \mathcal{C} \tag{6.2}$$

The cartesian lift of a given arrow $f : A \rightarrow G(1)$ is given by a natural transformation $f_G : f^*G \Rightarrow G$ obtained from the pullback square

$$\begin{array}{ccc} f^*GX & \xrightarrow{f_{G,X}} & GX \\ \downarrow & \lrcorner & \downarrow G! \\ A & \xrightarrow{f} & G1 \end{array} \tag{6.3}$$

that simultaneously defines f^*G (on morphisms is defined by pullback again) and f_G .

The fibred subcategory of polynomial functors is what gives ‘dependent’ lenses, whose opposite is the codomain fibration, i.e. ‘dependent’ charts [Spi19]. This suggests that taking the opposite fibration of $-(1)$ gives us a fibration of ‘generalized charts’.

We can explicitly construct these things if we work out the cartesian factorization system induced by $-(1)$ on $\mathbf{End}(\mathcal{C})$. This is given by

- (1) Cartesian maps are given by natural transformations whose naturality is witness by pullback squares, as suggested by the definition of f^*G above: which we make explicit here:

$$\begin{array}{ccc} FX & \xrightarrow{\alpha_X} & GX \\ Ff \downarrow & \lrcorner & \downarrow Gf \\ FY & \xrightarrow{\alpha_Y} & GY \end{array} \quad (6.4)$$

- (2) Vertical maps are given by natural transformations whose component at 1 is an isomorphism (think: the identity)

We define a *generalized chart* $\left(\begin{smallmatrix} k^b \\ k \end{smallmatrix}\right) : F \rightrightarrows G$ to be a span in $\text{End}(\mathcal{C})$ whose left leg is vertical and whose right leg is cartesian. Generalized charts look like this:

$$\begin{array}{ccccc} \text{End}(\mathcal{C}) & & F & \xleftarrow{k^b} & k^*G & \xrightarrow{k_G} & G \\ \downarrow -(1) & & & & & & \\ \mathcal{C} & & F1 & \xlongequal{\quad} & F1 & \xrightarrow[k]{} & G1 \end{array} \quad (6.5)$$

These might look like lenses (because lenses are obtained by opping a fibration) but they are actually charts. We can verify this by looking at the case in which F and G are polynomial (over **Set**), to see if this construction recovers the usual one. We see that k^b lives in

$$\text{Nat} \left(\sum_{i \in F1} y^{F_i}, \sum_{j \in F1} y^{G_{k(j)}} \right) \cong \prod_{i \in F1} \sum_{j \in F1} \mathbf{Set}(F_i, G_{k(j)}) \cong \sum_{f: F1 \rightarrow F1} \prod_{i \in F1} \mathbf{Set}(F_i, G_{k(f(i))}) \quad (6.6)$$

and since we know it is a vertical map, then k^b actually lives in the subobject of the right hand side for which the map f is the identity. Thus we see $\left(\begin{smallmatrix} k^b \\ k \end{smallmatrix}\right)$ encodes the data of a chart (notice how F and G swapped places: charts are lenses ‘relative to lenses’).

This allows us to extend the indexed category **Coalg** defined previously to have a profunctorial action. So a given generalized chart $\left(\begin{smallmatrix} k^b \\ k \end{smallmatrix}\right) : F \rightrightarrows G$ is mapped to a profunctor $\mathbf{Coalg}\left(\begin{smallmatrix} k^b \\ k \end{smallmatrix}\right) : \mathbf{Coalg}(F) \leftrightarrow \mathbf{Coalg}(G)$. This has a rather complex definition: it maps two coalgebras $\gamma : A \rightarrow FA$ and $\delta : B \rightarrow GB$ to the set of $\phi : A \rightarrow B$ that make the following commute:

$$\begin{array}{ccccc} A & & & & B \\ \downarrow \gamma & & \downarrow \delta' & \lrcorner & \downarrow \delta \\ & k^*B & \xrightarrow{k'} & & B \\ & \downarrow \delta' & & & \downarrow \delta \\ & k^*GB & \xrightarrow{k_{G,B}} & & GB \\ & \downarrow G! & & & \downarrow G! \\ & F1 & \xrightarrow{k} & & G1 \end{array} \quad (6.7)$$

(Note: In the original image, there is a curved dashed arrow from A to B labeled ϕ , and a curved arrow from FA to $F1$ labeled $F!$. The square between k^*B and k^*GB is also indicated with a $\lrcorner$ symbol.)

$$\begin{array}{ccccc} A & \xrightarrow{\quad \exists! \langle \gamma, F!, \phi \rangle \quad} & k^*B \\ \downarrow \gamma & & \downarrow \delta' \\ FA & \xrightarrow{F(\phi)} & FB & \xleftarrow{k_B^b} & k^*GB \end{array} \quad (6.8)$$

Specifically, we are asking for the following:

$$\begin{aligned} \forall a \in A, \quad G!(\delta(\phi(a))) &= k(F!(\gamma(a))) \\ \forall a \in A, \quad k_B^b(\delta'(F!(\gamma(a)), \phi(a))) &= F(\phi)(\gamma(a)) \end{aligned} \tag{6.9}$$

Matteo: The double category $\text{End} \dots$

All in all, this gives us the **theory of coalgebras**:

$$\mathbf{Coalg} : \mathbf{End}^\top \xrightarrow{\text{unitary lax}} \mathbf{Cat}. \tag{6.10}$$

REFERENCES

- [AC04] S. Abramsky and B. Coecke. “A categorical semantics of quantum protocols”. In: *Proceedings of the 19th Annual IEEE Symposium on Logic in Computer Science, 2004*. IEEE. 2004, pp. 415–425.
- [AM74a] M. A. Arbib and E. G. Manes. “Foundations of System Theory: Decomposable Systems”. In: *Automatica* 10.3 (May 1, 1974), pp. 285–302. ISSN: 0005-1098. DOI: [10.1016/0005-1098\(74\)90039-9](https://doi.org/10.1016/0005-1098(74)90039-9). URL: <https://www.sciencedirect.com/science/article/pii/0005109874900399> (visited on 11/06/2023).
- [AM74b] M. A. Arbib and E. G. Manes. “Machines in a Category: An Expository Introduction”. In: *SIAM Review* 16.2 (Apr. 1974), pp. 163–192. ISSN: 0036-1445. DOI: [10.1137/1016026](https://doi.org/10.1137/1016026). URL: <https://epubs.siam.org/doi/abs/10.1137/1016026> (visited on 11/06/2023).
- [AM75] M. A. Arbib and E. G. Manes. “Adjoint Machines, State-Behavior Machines, and Duality”. In: *Journal of Pure and Applied Algebra* 6.3 (Nov. 1, 1975), pp. 313–344. ISSN: 0022-4049. DOI: [10.1016/0022-4049\(75\)90028-6](https://doi.org/10.1016/0022-4049(75)90028-6). URL: <https://www.sciencedirect.com/science/article/pii/0022404975900286> (visited on 11/06/2023).
- [AZ69] M. A. Arbib and H. P. Zeiger. “On the Relevance of Abstract Algebra to Control Theory”. In: *Automatica* 5.5 (Sept. 1, 1969), pp. 589–606. ISSN: 0005-1098. DOI: [10.1016/0005-1098\(69\)90026-0](https://doi.org/10.1016/0005-1098(69)90026-0). URL: <https://www.sciencedirect.com/science/article/pii/0005109869900260> (visited on 11/06/2023).
- [BC20] J. C. Baez and K. Courser. “Structured cospans”. In: *Theory and Applications of Categories* 35.48 (2020), pp. 1771–1822.
- [BCV22] J. C. Baez, K. Courser, and C. Vasilakopoulou. “Structured versus Decorated Cospans”. In: *Compositionality* 4 (3 Sept. 2022). ISSN: 2631-4444. DOI: [10.32408/compositionality-4-3](https://doi.org/10.32408/compositionality-4-3). URL: <https://doi.org/10.32408/compositionality-4-3>.
- [BM20] J. C. Baez and J. Master. “Open petri nets”. In: *Mathematical Structures in Computer Science* 30.3 (2020), pp. 314–341.
- [BSW96] S. L. Bloom, N. Sabadini, and R. F. Walters. “Matrices, machines and behaviors”. In: *Applied Categorical Structures* 4.4 (1996). Publisher: Springer, pp. 343–360.
- [Cap22] M. Capucci. *From categorical systems theory to categorical cybernetics*. 2022. URL: <https://www.youtube.com/watch?v=wtgfyjFIHBQ>.

- [CEG+20] B. Clarke et al. “Profunctor optics, a categorical update”. In: *arXiv preprint arXiv:2001.07488* (2020).
- [CGKS20] J. Culbertson et al. “Formal composition of hybrid systems”. In: *Theory and Applications of Categories* 35.45 (2020), pp. 1634–1682.
- [CK18] B. Coecke and A. Kissinger. “Picturing quantum processes”. In: *International Conference on Theory and Application of Diagrams*. Springer. 2018, pp. 28–31.
- [Cou20] K. Courser. *Open Systems: A Double Categorical Perspective*. Aug. 5, 2020. DOI: [10.48550/arXiv.2008.02394](https://doi.org/10.48550/arXiv.2008.02394). arXiv: [2008.02394 \[math\]](https://arxiv.org/abs/2008.02394). URL: <http://arxiv.org/abs/2008.02394> (visited on 05/10/2024). Pre-published.
- [DGR+21] E. Di Lavore et al. “A canonical algebra of open transition systems”. In: *International Conference on Formal Aspects of Component Software*. Springer. 2021, pp. 63–81.
- [Fon15] B. Fong. “Decorated Cospans”. In: *Theory & Applications of Categories* 30 (2015).
- [FS07] J. L. Fiadeiro and V. Schmitt. “Structured co-spans: an algebra of interaction protocols”. In: *International Conference on Algebra and Coalgebra in Computer Science*. Springer. 2007, pp. 194–208.
- [Gog72a] J. A. Goguen. “Minimal Realization of Machines in Closed Categories”. In: *Bulletin of the American Mathematical Society* 78.5 (Sept. 1, 1972), pp. 777–783. DOI: [10.1090/s0002-9904-1972-13032-5](https://doi.org/10.1090/s0002-9904-1972-13032-5).
- [Gog72b] J. A. Goguen. “Realization Is Universal”. In: *Theory of Computing Systems \ Mathematical Systems Theory* 6.4 (Dec. 1, 1972), pp. 359–374. DOI: [10.1007/bf01843493](https://doi.org/10.1007/bf01843493).
- [Gog73] J. Goguen. “Categorical Foundations for General Systems Theory”. In: *Advances in cybernetics and systems research* 1 (1973).
- [Gra19] M. Grandis. *Higher Dimensional Categories: From Double to Multiple Categories*. World Scientific, 2019.
- [Hed23] J. Hedges. *Geometry of interaction is the optic for copointed functors*. en-GB. Jan. 2023. URL: <https://julesh.com/2023/01/28/geometry-of-interaction-is-the-optic-for-copointed-functors/> (visited on 03/03/2023).
- [Jac17] B. Jacobs. *Introduction to coalgebra*. Vol. 59. Cambridge University Press, 2017. URL: <https://books.google.com/books?hl=en&lr=&id=tApQDQAAQBAJ&oi=fnd&pg=PR7&dq=jacobs+introduction+to+coalgebra&ots=Zm3dpD7Sbo&sig=R5uGNh6AARnrbrRu7r04006t7As> (visited on 10/30/2023).
- [Jac99] B. Jacobs. *Categorical logic and type theory*. 1st. Vol. 141. Studies in Logic and the Foundations of Mathematics. Elsevier, Jan. 1999. ISBN: 978-0-444-50853-9.
- [Koc22] J. Kock. “Whole-grain Petri Nets and Processes”. In: *Journal of the ACM* 70.1 (Dec. 2022), 1:1–1:58. ISSN: 0004-5411. DOI: [10.1145/3559103](https://doi.org/10.1145/3559103). URL: <https://doi.org/10.1145/3559103> (visited on 12/10/2023).
- [KSW02] P. Katis, N. Sabadini, and R. F. Walters. “Feedback, trace and fixed-point semantics”. In: *RAIRO-Theoretical Informatics and Applications* 36.2 (2002), pp. 181–194.
- [KSW97a] P. Katis, N. Sabadini, and R. F. Walters. “Bicategories of processes”. In: *Journal of Pure and Applied Algebra* 115.2 (1997), pp. 141–178.

- [KSW97b] P. Katis, N. Sabadini, and R. F. Walters. “Span(Graph): A categorical algebra of transition systems”. In: *International Conference on Algebraic Methodology and Software Technology*. Springer. 1997, pp. 307–321.
- [KSW99] P. Katis, N. Sabadini, and R. F. Walters. *On the algebra of feedback and systems with boundary*. Citeseer, 1999.
- [KV08] C. Kupke and Y. Venema. “Coalgebraic Automata Theory: Basic Results”. en. In: *Logical Methods in Computer Science* Volume 4, Issue 4 (Nov. 2008), p. 1203. ISSN: 1860-5974. DOI: [10.2168/LMCS-4\(4:10\)2008](https://doi.org/10.2168/LMCS-4(4:10)2008). URL: <https://lmcs.episciences.org/1203> (visited on 10/30/2023).
- [LBPF21] S. Libkind et al. “Operadic modeling of dynamical systems: mathematics and computation”. In: (2021). Available at <https://arxiv.org/abs/2105.12282>.
- [Lei04] T. Leinster. *Higher operads, higher categories*. 298. Cambridge University Press, 2004.
- [Ler18] E. Lerman. “Networks of open systems”. In: *Journal of Geometry and Physics* 130 (2018), pp. 81–112.
- [Mos99] L. S. Moss. “Coalgebraic Logic”. In: *Annals of Pure and Applied Logic* 96.1 (Mar. 1, 1999), pp. 277–317. ISSN: 0168-0072. DOI: [10.1016/S0168-0072\(98\)00042-6](https://doi.org/10.1016/S0168-0072(98)00042-6). URL: <https://www.sciencedirect.com/science/article/pii/S0168007298000426> (visited on 05/26/2024).
- [Mye20a] D. J. Myers. *A general definition of open dynamical system*. 2020. URL: <https://www.youtube.com/watch?v=8T-Km3taNko>.
- [Mye20b] D. J. Myers. *Double Categories of Open Dynamical Systems*. 2020. URL: https://www.youtube.com/watch?v=f9fjf9lo2_M.
- [Mye20c] D. J. Myers. “Double categories of open dynamical systems”. In: *arXiv preprint arXiv:2005.05956* (2020).
- [Mye20d] D. J. Myers. *Open dynamical systems, trajectories and hierarchical planning*. 2020. URL: <https://www.youtube.com/watch?v=3FxeY5DbPn0>.
- [Mye21] D. J. Myers. *Paradigms of composition*. 2021. URL: <https://www.youtube.com/watch?v=50s62D5Ah-M>.
- [Mye22] D. J. Myers. *Categorical Systems Theory*. 2022.
- [Rom20] M. Román. “Profunctor optics and traversals”. In: *arXiv preprint arXiv:2001.08045* (2020).
- [Ros78] R. Rosen. *Fundamentals of measurement and representation of natural systems*. Vol. 1. Elsevier Science Limited, 1978.
- [Rut00] J. J. Rutten. “Universal coalgebra: a theory of systems”. In: *Theoretical computer science* 249.1 (2000). Publisher: Elsevier, pp. 3–80. URL: <https://www.sciencedirect.com/science/article/pii/S0304397500000566> (visited on 10/30/2023).
- [Spi13] D. I. Spivak. “The operad of wiring diagrams: formalizing a graphical language for databases, recursion, and plug-and-play circuits”. In: (2013). Available at <https://arxiv.org/abs/1305.0297>.
- [Spi19] D. I. Spivak. “Generalized Lens Categories via functors $F : \mathcal{C}^{\text{op}} \rightarrow \mathbf{Cat}$ ”. 2019.

- [Spi20] D. I. Spivak. *Poly: An abundant categorical setting for mode-dependent dynamics*. arXiv:2005.01894 [math]. June 2020. DOI: [10.48550/arXiv.2005.01894](https://doi.org/10.48550/arXiv.2005.01894). URL: <http://arxiv.org/abs/2005.01894> (visited on 03/03/2023).
- [SW93] N. Sabadini and R. F. Walters. “On functions and processors: an automata-theoretic approach to concurrency through distributive categories”. In: *School of Mathematics and Statistics Research Reports* 7 (93 Nov. 1993).
- [TP97] D. Turi and G. Plotkin. “Towards a mathematical operational semantics”. In: *Proceedings of Twelfth Annual IEEE Symposium on Logic in Computer Science*. ISSN: 1043-6871. June 1997, pp. 280–291. DOI: [10.1109/LICS.1997.614955](https://doi.org/10.1109/LICS.1997.614955).
- [Trn77] V. Trnková. “Relational automata in a category and their languages”. In: *Fundamentals of Computation Theory*. Ed. by G. Goos et al. Vol. 56. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 1977, pp. 340–355. ISBN: 978-3-540-08442-6 978-3-540-37084-0. DOI: [10.1007/3-540-08442-8_101](https://doi.org/10.1007/3-540-08442-8_101). URL: http://link.springer.com/10.1007/3-540-08442-8_101 (visited on 10/30/2023).
- [VSL14] D. Vagner, D. I. Spivak, and E. Lerman. “Algebras of open dynamical systems on the operad of wiring diagrams”. In: (2014). Available at <https://arxiv.org/abs/1408.1598>.